Lezioni Algoritmi e Strutture Dati

Alberto Bernasconi

a.a 25/26

Indice

1 Problemi Emblematici, Efficienza e Risorse **2**

2 Formalizzazione dei Problemi e Analisi della Complessità **7**

3 Notazioni Asintotiche e Modelli di Calcolo Teorici **10**

4 Modelli di Calcolo Teorici **14**

5 Macchina RAM e Criteri di Costo Computazionale **16**

6 Limiti RAM, Macchina RASP e Calcolabilità **19**

7 Linguaggi ad Alto Livello e Modularità **24**

8 Strutture Dati: Dalla Memoria Statica all'Allocazione Dinamica **27**

9 Strutture Dati Elementari e Mappe di Memorizzazione **29**

10 Strutture Dati Dinamiche: Liste e Accesso Ristretto **34**

11 Algoritmi su Liste Dinamiche e Strutture LIFO **39**

12 Implementazione delle Strutture Dati e Teoria dei Grafi **43**

13 Alberi: Topologia, Proprietà Matematiche e Combinatoria **48**

14 Proprietà degli Alberi e Attraversamento dei Grafi **52**

15 Esplorazione di Grafi e Alberi: Strategie e Complessità **56**

16 Visita Iterativa di Alberi e Problema dell'Ordinamento **60**

Capitolo 1

Problemi Emblematici, Efficienza e Risorse

23/02/2026 **Il Problema della Celebrità**

Output 1: Diario Generale di Ripasso (Formato Accademico)

- **Struttura di un problema:**
- **Partiamo da una definizione formale:**
  - **Celebrità**[1]
  - **Problema**[2]
  - **Vincolo Logico**[3]
  - **Soluzione Naive**[4]
- **La soluzione:**
  - Selezionare una coppia casuale di individui (X, Y) e porre la domanda: "X conosce Y?".
  - Analisi delle risposte: se la risposta è affermativa, X non può essere la celebrità; se la risposta è negativa, Y non può essere la celebrità.
  - In ogni caso, una singola domanda elimina un candidato. Ripetendo il processo su un insieme decrescente, dopo $n-1$ domande rimane un unico potenziale candidato.
  - Si verifica infine se il candidato rimasto soddisfa le condizioni su tutti gli altri membri. Il numero massimo di domande necessarie è descritto dalla seguente equazione di ricorrenza:

$$c_n = c_{n-1} + 1, \quad c_1 = 2n - 3$$

[1]**Celebrità:** Una persona che non conosce nessuno all'interno di un gruppo, ma che è conosciuta da tutti gli altri membri del medesimo gruppo.

[2]**Problema:** Dato un insieme di n persone, determinare se questo contiene una celebrità ponendo esclusivamente domande del tipo "X conosce Y?".

[3]**Vincolo Logico:** All'interno di un gruppo può esistere al massimo una singola celebrità (dimostrabile per assurdo per l'asimmetria della conoscenza).

[4]**Soluzione Naive:** Esaminare ogni persona e confrontarla con tutte le altre $n-1$ persone per verificare la duplice condizione (non conoscere nessuno ed essere nota a tutti). Richiede $2n(n-1)$ domande in totale.

Sviluppando l'equazione si ottiene:

$$c_n = \underbrace{1 + 1 + \cdots + 1}_{n-1 \text{ volte}} + 2n - 3 = n - 1 + 2n - 3 = 3n - 4$$

La complessità del problema passa quindi da un andamento quadratico $\Theta(n^2)$ a un andamento lineare $\Theta(n)$.

Guida di Studio Approfondita e Semplificata (Formato Apprendimento)

Il Concetto Base: Immagina di essere a una festa con 100 persone e di dover trovare una star di fama mondiale che però vive isolata e non conosce nessuno dei presenti. Il metodo più banale è chiedere a tutti se conoscono tutti: un'infinità di domande! Ma c'è un trucco: ogni volta che chiedi "Scusa, tu conosci quella persona?", se la risposta è "Sì", chi ha risposto non è la star. Se la risposta è "No", l'altra persona non è la star. Ogni singola domanda elimina matematicamente una persona dalla lista dei sospettati.

Analisi Passo-Passo: L'algoritmo si divide in due fasi distinte:

1. *Fase di Eliminazione:* Si prendono due persone a caso, si fa la domanda, e si scarta chi fallisce il test. Si ripete fino a quando non rimane un solo candidato. Questa fase costa esattamente $n - 1$ domande.

2. *Fase di Verifica:* Il candidato rimasto potrebbe benissimo essere una persona normale (se la celebrità non è presente alla festa). Si procede quindi a testare quest'unico candidato contro il resto del gruppo per confermare che effettivamente non conosca nessuno e sia noto a tutti.

Il "Perché" è importante: Questo problema dimostra il potere dell'ottimizzazione algoritmica. Passare da $2n(n-1)$ domande a $3n - 4$ domande significa che, su 100.000 persone, invece di fare quasi 20 miliardi di domande (con costi enormi in termini di tempo e risorse), ne bastano meno di 300.000. In informatica, abbassare l'ordine di grandezza della complessità (da quadratico a lineare) è fondamentale per rendere un problema risolvibile nella pratica.

Ripasso Visivo/Schematico:

Approccio	**Strategia**	**Numero di Domande**
Naive	Interrogazione totale	$2n(n-1)$ (Quadratico)
Efficiente	Eliminazione e successiva verifica	$3n - 4$ (Lineare)

23/02/2026 Il Problema del Missionario Chirurgo

Output 1: Diario Generale di Ripasso (Formato Accademico)

- **Struttura di un problema:**
- **Partiamo da una definizione formale:**
  - **Problema**[5]

[5] **Problema:** Determinare il numero massimo di pazienti che un chirurgo può operare in totale sicurezza avendo a disposizione solamente n paia di guanti sterili.

- **Vincolo Operativo**[6]
- **Limite Teorico**[7]
- **Soluzione Naive**[8]

- **La soluzione:**
  - Per raggiungere il limite teorico di $2n - 1$ operazioni, si sfrutta il principio della sovrapposizione e della reversibilità.
  - Il chirurgo indossa tutti gli n paia di guanti uno sopra l'altro ed effettua il primo intervento.
  - Al termine, sfila il paio più esterno (contaminato all'esterno, sterile all'interno), lo accantona e procede ad effettuare interventi con gli $n - 1$ guanti rimanenti.
  - Una volta esauriti i guanti, si recuperano i guanti accantonati: vengono rovesciati e re-indossati. La parte precedentemente a contatto con i tessuti (ora interna) va a toccare la superficie esterna infetta dei guanti già indossati, mantenendo sterile la nuova superficie esterna.
  - Il numero totale di interventi risolve la seguente equazione di ricorrenza:

    $$c_n = c_{n-1} + 2, \quad c_1 = 1$$

    Sviluppando iterativamente:

    $$c_n = \underbrace{2 + 2 + \cdots + 2}_{n-1 \text{ volte}} + 1 = 2(n-1) + 1 = 2n - 1$$

Guida di Studio Approfondita e Semplificata (Formato Apprendimento)

Il Concetto Base: Se hai 10 paia di guanti, l'istinto dice che puoi fare 10 operazioni. Ma le risorse sono scarse e devi massimizzarne l'uso. Ogni guanto ha un "dentro" e un "fuori". Se consideri il totale delle superfici, hai 20 facce utilizzabili. Togliendone una che inevitabilmente si sporcherà toccando la mano del chirurgo, ne restano 19. Il problema diventa: come sfruttare tutte queste 19 facce senza mescolare lo sporco?

Analisi Passo-Passo: Il trucco è la stratificazione.

1. Metti tutti i guanti assieme: operi e ne togli uno. La faccia esterna del guanto rimanente è perfettamente pulita.
2. Continui a togliere strati operando ogni volta. Questo ti garantisce n operazioni.
3. I guanti tolti sono sporchi fuori ma pulitissimi dentro. Se li giri al rovescio, il "pulito" va fuori. Lo sporco va dentro, ma toccando un guanto già sporco che hai sulla mano, non contamina nulla di critico.
4. Riempiendoti le mani al contrario con i guanti girati, fai altre $n - 1$ operazioni.

[6] **Vincolo Operativo:** Per garantire la sicurezza, non deve esserci alcun contatto tra i tessuti infetti dei pazienti e le superfici dei guanti (o la mano del chirurgo) in modo crociato.

[7] **Limite Teorico:** Ogni paio di guanti dispone di due superfici (interna ed esterna). Con n paia, vi sono $2n$ superfici totali. Poiché almeno una superficie deve essere a contatto con la mano del chirurgo, restano al massimo $2n - 1$ superfici utilizzabili per i pazienti.

[8] **Soluzione Naive:** Utilizzare un nuovo paio di guanti per ogni operazione, garantendo un totale di n interventi sicuri.

Il "Perché" è importante: Dal punto di vista algoritmico, questo problema introduce un concetto duale rispetto a quello della celebrità. Lì l'obiettivo era *minimizzare* le operazioni (il tempo/le domande) dato un input fisso. Qui le risorse sono fisse (n guanti) e l'obiettivo è *massimizzare* il risultato (il numero di operazioni). Sfruttando le proprietà fisiche del problema (topologia delle superfici), l'algoritmo raggiunge l'ottimo teorico assoluto.

Ripasso Visivo/Schematico:

Fase	**Azione Meccanica**	**Interventi Cumulati**
Andata	Sfilare i guanti stratificati uno ad uno	n
Ritorno	Rovesciare e re-indossare i guanti usati	$n-1$
Totale		$\mathbf{2n-1}$

23/02/2026 Equa Divisione (Fair Division)

Output 1: Diario Generale di Ripasso (Formato Accademico)

- **Struttura di un problema:**
- **Partiamo da una definizione formale:**
  - **Problema (Fair Division)**[9]
  - **Vincolo 1**[10]
  - **Vincolo 2**[11]
  - **Vincolo 3**[12]
- **La soluzione:**
  - **Algoritmo per $k = 2$:**
    1. Il soggetto A divide l'insieme S in due partizioni: $S = S_1 \cup S_2$.
    2. Il soggetto B ha il diritto di prima scelta tra S_1 e S_2. A riceve l'insieme rimanente.
  - **Algoritmo per $k = 3$:**
    1. Il soggetto A crea una partizione in tre sottoinsiemi: $S = S_1 \cup S_2 \cup S_3$.
    2. Il soggetto B è chiamato a indicare l'insieme S_i che, secondo la sua metrica di valutazione, ha il valore strettamente minore (cioè $Val(S_i) \leq R/3$).
    3. Interviene il soggetto C valutando l'insieme S_i scelto da B:
       * Se C concorda con B valutando $Val(S_i) \leq R/3$, allora l'insieme S_i viene forzatamente assegnato ad A. A questo punto, il problema si riduce a dividere l'insieme rimanente $S \backslash S_i$ tra i soggetti B e C utilizzando l'algoritmo per $k = 2$.
       * Se C non concorda e ritiene che $Val(S_i) > R/3$, allora l'insieme S_i viene assegnato a C. Il problema si riduce a dividere $S \setminus S_i$ tra A e B utilizzando l'algoritmo per $k = 2$.

[9] **Problema (Fair Division):** k individui devono spartirsi in modo equo un insieme S di oggetti aventi un valore soggettivo complessivo pari a R.

[10] **Vincolo 1:** Non deve essere possibile la formazione di coalizioni a danno di un singolo individuo.

[11] **Vincolo 2:** Se un individuo A riceve un sottoinsieme S_1 con valore $Val(S_1) < R/k$, la responsabilità di tale deficit deve ricadere esclusivamente sulle scelte logiche di A.

[12] **Vincolo 3:** Assenza di invidia (Envy-Free): L'individuo A non può invidiare le assegnazioni altrui se l'insieme S_1 a lui assegnato soddisfa la condizione $Val(S_1) \geq R/k$.

Guida di Studio Approfondita e Semplificata (Formato Apprendimento)

Il Concetto Base: È il classico problema del "dividere la torta". Se due bambini devono dividersi una torta, il modo più pacifico è far tagliare la fetta al primo bambino e far scegliere al secondo. Il primo taglierà le fette il più precisamente possibile a metà per non rischiare che l'altro gli porti via la porzione più grande. Questo principio si estende matematicamente a gruppi più grandi.

Analisi Passo-Passo: Quando si passa a tre persone (A, B, C), le cose si complicano. A taglia in tre parti. A questo punto B punta il dito contro la fetta che reputa più "scarsa". Qui scatta la trappola logica affidata a C:

1. Se C è d'accordo che quella fetta fa schifo, viene data ad A. Perché? Perché A ha tagliato la torta! Se ha fatto una fetta così piccola, è solo colpa sua. A questo punto B e C si dividono ciò che resta con il metodo dei due bambini.

2. Se invece C pensa che quella fetta sia in realtà molto grossa (i valori sono puramente soggettivi!), allora C la prende felicemente, e A e B si dividono il resto.

Il "Perché" è importante: L'algoritmo dimostra come regole procedurali rigide possano forzare comportamenti onesti anche in entità puramente egoistiche. In ambito informatico ed economico, algoritmi di questo genere sono alla base della teoria dei giochi e della gestione dei conflitti nell'allocazione di risorse condivise (tempo di CPU, banda di rete, divisione di beni).

Ripasso Visivo/Schematico:

Soggetti	**Fase**	**Azione**
$k = 2$	*Taglio*	A divide in due.
	Scelta	B sceglie per primo, A prende il resto.
$k = 3$	*Taglio*	A divide in tre.
	Selezione	B isola la porzione peggiore S_i.
	Verifica di C	Se per C S_i è scarsa $\rightarrow$ Assegnata ad A.
		Se per C S_i è ricca $\rightarrow$ Assegnata a C.

Capitolo 2

Formalizzazione dei Problemi e Analisi della Complessità

24/02/2026 **Formalizzazione dei Problemi e Correttezza**

Output 1: Diario Generale di Ripasso (Formato Accademico)

- **Struttura di un problema:**
- **Partiamo da una definizione formale:**
  - **Problema**[1]
  - **Dimensione dell'istanza**[2]
  - **Algoritmo Corretto**[3]
  - **Esecutore**[4]
- **La soluzione:**
  - La valutazione di un algoritmo richiede di misurarne l'efficienza in termini di tempo e spazio.
  - Per slegare l'analisi dalla tecnologia hardware sottostante, il tempo non viene misurato in secondi, ma come *numero di istruzioni elementari eseguite* dalla macchina.
  - Esiste un'asimmetria logica tra le risorse di Spazio e Tempo:

    $$\text{Tempo Finito} \implies \text{Spazio Finito}$$

    In un numero finito di istruzioni è impossibile indirizzare e utilizzare un numero infinito di celle di memoria. Al contrario, uno Spazio Finito *non* implica necessariamente un Tempo Finito, in quanto la macchina potrebbe entrare in un loop infinito utilizzando sempre gli stessi registri.

[1] **Problema:** Mappatura tra un dominio di istanze in ingresso (dati) e un codominio di risposte (soluzioni). A ogni problema Π è associata una funzione formale $f_\Pi(x)$.

[2] **Dimensione dell'istanza:** Misura quantitativa dell'input, indicata tipicamente con n (es. numero di cifre, ordine di una matrice, numero di elementi in un vettore).

[3] **Algoritmo Corretto:** Un algoritmo si dice corretto se e solo se, per ogni istanza del problema, la funzione calcolata dall'algoritmo $\Phi_{ALG}(x)$ coincide esattamente con la funzione soluzione del problema $f_\Pi(x)$. L'insieme degli ingressi deve coincidere con il dominio del problema e l'insieme delle uscite con le soluzioni attese.

[4] **Esecutore:** Modello astratto capace di comprendere ed eseguire le istruzioni elementari di un algoritmo. Definire l'esecutore è fondamentale per quantificare oggettivamente il costo computazionale.

Guida di Studio Approfondita e Semplificata (Formato Apprendimento)

Il Concetto Base: Per scrivere un algoritmo efficiente, prima di tutto bisogna capire cos'è un "problema" in informatica. Immaginate il test di primalità: il dominio d'ingresso è l'insieme dei numeri naturali, e l'uscita desiderata è semplicemente un "Vero" o "Falso" (1 o 0). L'algoritmo è una macchina che trasforma l'input in output. Se l'algoritmo non fa esattamente quello che richiede il problema per *ogni* singolo input, allora non è un algoritmo corretto.

Analisi Passo-Passo: Quando misuriamo l'efficienza di un algoritmo corretto, perché non usiamo semplicemente un cronometro? Perché se usassi un cronometro sul mio vecchio PC impiegherei 10 secondi, ma su un supercomputer impiegherei un millesimo di secondo. L'algoritmo è sempre lo stesso! Per questo motivo, in informatica teorica si conta il *numero di operazioni base* che l'algoritmo esegue. Questo numero dipenderà dalla "dimensione" del problema (es. quanto è grande il numero da analizzare, quanti elementi ci sono in un vettore).

Il "Perché" è importante: Concentrarsi sul numero di istruzioni piuttosto che sui secondi ci fornisce una valutazione oggettiva e universale. Sapere che un tempo finito garantisce uno spazio di memoria finito ci aiuta a capire che la risorsa più critica da limitare è solitamente il tempo di esecuzione, piuttosto che la disponibilità infinita di registri teorici.

Ripasso Visivo/Schematico:

Concetto	**Descrizione Operativa**
Dominio In/Out	Deve esserci corrispondenza biunivoca tra le istanze del problema e gli ingressi dell'algoritmo.
Misura del Tempo	Indipendente dall'hardware (Conta il numero di istruzioni).
Spazio/Tempo	Tempo Finito $\implies$ Spazio Finito (non vale l'inverso).

24/02/2026 Tipologie di Complessità e Impatto Tecnologico

Output 1: Diario Generale di Ripasso (Formato Accademico)

- **Struttura di un problema:**
- **Partiamo da una definizione formale:**
  - **Complessità al Caso Peggiore (T^w)**[5]
  - **Complessità al Caso Migliore (T^b)**[6]
  - **Complessità in Media (T^a)**[7]
- **La soluzione:**
  - L'analisi in media richiede la conoscenza della distribuzione di probabilità delle istanze. *(Nota: le registrazioni mostrano frammentarietà nella trattazione delle derivazioni statistiche complete).* Sotto l'ipotesi di *equiprobabilità* delle istanze, la probabilità diventa uniformemente $p_i = \frac{1}{|I_n|}$, e la formula diviene:

$$T^a(n) = \sum_k k \cdot \frac{\sigma_{n,k}}{|I_n|}$$

[5] **Complessità al Caso Peggiore (T^w):** Il limite superiore delle risorse (tempo o spazio) necessarie per elaborare l'istanza più sfavorevole di dimensione n.

[6] **Complessità al Caso Migliore (T^b):** Il limite inferiore delle risorse richieste, associato all'istanza più favorevole di dimensione n.

[7] **Complessità in Media (T^a):** Il comportamento atteso dell'algoritmo, calcolato come la somma pesata delle risorse necessarie per ciascuna istanza, moltiplicata per la sua probabilità di occorrenza.

dove $\sigma_{n,k}$ è il numero di istanze di dimensione n che richiedono costo k.

- **L'impatto dell'hardware sulla complessità:** Il progresso tecnologico (es. processori 10 volte più veloci) ha impatti drastici in base all'algoritmo scelto.
- Se l'algoritmo è Lineare $\Theta(n)$: un esecutore 10x più veloce permette di risolvere istanze 10 volte più grandi.
- Se l'algoritmo è Esponenziale $\Theta(2^n)$: se la vecchia istanza massima era $n_{old} = \log_2(C)$, la nuova sarà:

$$n_{new} = \log_2(10C) = \log_2(10) + \log_2(C) \approx 3.32 + n_{old}$$

Un incremento puramente additivo e trascurabile.

Guida di Studio Approfondita e Semplificata (Formato Apprendimento)

Il Concetto Base: Per valutare un algoritmo si usano tre metri di giudizio:

- *Caso Peggiore:* È il "conto massimo" al ristorante. Serve se l'applicazione è critica. Se programmi i sensori di una centrale nucleare, non ti interessa il caso medio in cui va tutto bene; devi essere certo che anche nell'istanza più sfortunata l'algoritmo finisca in tempo per evitare l'esplosione.
- *Caso Migliore:* Quando ti va tutto liscio. È utile, ma raramente rappresenta un vincolo ingegneristico di design.
- *Caso Medio:* Il comportamento normale. Utile nei bancomat o nelle code al supermercato, dove si accetta che raramente (nel caso peggiore) un utente possa dover aspettare a lungo (es. va in time-out), purché la maggior parte dei clienti sia servita velocemente.

Analisi Passo-Passo: Perché studiare l'algoritmica invece di aspettare semplicemente computer più veloci? L'analisi mostra matematicamente che l'hardware potente non salva i cattivi algoritmi. Se hai un algoritmo esponenziale (2^n) che in un'ora gestisce 25 dati, comprando un PC che va ben 10 volte più veloce non riuscirai a gestire 250 dati... ne gestirai solo 28! La crescita esponenziale consuma istantaneamente tutta la potenza extra. Con un algoritmo efficiente (es. lineare), invece, un PC 10 volte più veloce ti fa elaborare esattamente 10 volte più dati.

Il "Perché" è importante: Il design del software (l'algoritmo) domina sempre l'hardware. Gli algoritmi esponenziali o troppo inefficienti rendono inapplicabili certe soluzioni indipendentemente dalla velocità dei processori futuri (fino a limiti fisici o quantistici). Questa è la base per la ricerca dell'ottimizzazione e delle euristiche.

Ripasso Visivo/Schematico:

Complessità	**Con PC 10x più veloce**	**Impatto sull'Istanza Risolvibile**
$O(n)$ Lineare	La dimensione cresce $10\times$	Massima scalabilità e ottimo guadagno
$O(n^2)$ Quadratica	La dimensione cresce $\approx \sqrt{10}\times$	Crescita tangibile (≈ 3.16 volte maggiore)
$O(2^n)$ Esponenziale	Aggiunge solo ≈ 3 elementi	Nessun guadagno reale ($+3.32$)

Capitolo 3

Notazioni Asintotiche e Modelli di Calcolo Teorici

26/02/2026 **Notazioni Asintotiche e Classi di Crescita**

Output 1: Diario Generale di Ripasso (Formato Accademico)

- **Struttura di un problema:** Semplificazione matematica per l'analisi rigorosa della complessità computazionale. L'obiettivo è ignorare costanti hardware-dipendenti e fattori additivi minori per determinare esclusivamente il comportamento dell'algoritmo al limite (per una dimensione dell'istanza $n \to \infty$).
- **Partiamo da una definizione formale:** Siano $f(n)$ e $g(n)$ funzioni definite sui numeri naturali a valori reali non negativi.
  - **Limite Superiore: O-grande (O)**[1]
  - **Limite Inferiore: Ω-grande (Ω)**[2]
  - **Stretto: Θ-grande (Θ)**[3]
  - **Equivalenza Asintotica ($\sim$) e o-piccolo (o)**[4]
- **La soluzione:** Per poter analizzare la complessità delle macrostrutture algoritmiche si definisce un'algebra su tali notazioni:
  - O e Ω soddisfano la proprietà riflessiva e transitiva.
  - Θ e $\sim$ sono relazioni di equivalenza (riflessive, simmetriche, transitive) e inducono una partizione nel mondo delle funzioni tramite classi di equivalenza.
  - L'algebra delle costanti definisce la conservazione dell'ordine:

$$c \cdot f(n) = O(g(n)), \quad \forall c > 0$$

[1]**Limite Superiore: O-grande (O):** Esistono un intero n_0 e una costante $c > 0$ per cui $\forall n > n_0, f(n) \leq c \cdot g(n)$. Informalmente, f è dominata o cresce al massimo quanto g.

[2]**Limite Inferiore: Ω-grande (Ω):** Esistono un intero n_0 e una costante $c > 0$ per cui $\forall n > n_0, f(n) \geq c \cdot g(n)$. Informalmente, f domina g, ossia l'algoritmo richiede *almeno* quel quantitativo di risorse.

[3]**Stretto: Θ-grande (Θ):** Esistono costanti positive c, d e un intero n_0 per cui $\forall n > n_0, c \cdot g(n) \leq f(n) \leq d \cdot g(n)$. Condizione necessaria e sufficiente: $f(n) = O(g(n)) \wedge f(n) = \Omega(g(n))$. Indica lo stesso ordine di grandezza.

[4]**Equivalenza Asintotica ($\sim$) e o-piccolo (o):** $f(n) \sim g(n) \iff \lim_{n\to\infty} \frac{f(n)}{g(n)} = 1$. $f(n) = o(g(n)) \iff \lim_{n\to\infty} \frac{f(n)}{g(n)} = 0$.

- Regole per la somma e prodotto: se $f_1(n) = O(g_1(n))$ e $f_2(n) = O(g_2(n))$, allora:

$$f_1(n) + f_2(n) = O(g_1(n) + g_2(n)) \quad \text{e} \quad f_1(n) \cdot f_2(n) = O(g_1(n) \cdot g_2(n))$$

- *Risoluzione del controesempio per la sottrazione (dimostrazione frammentaria trascritta dall'audio):* La sottrazione non preserva la notazione O-grande. Se si assume $f_1(n) = n^3 + n^2$ e $f_2(n) = n^3 + n$, è chiaro che entrambe appartengono a $O(n^3)$. Ma compiendo $f_1(n) - f_2(n) = n^2 - n$, che non appartiene a $O(n^3 - n^3) = O(0)$.

Guida di Studio Approfondita e Semplificata (Formato Apprendimento)

Il Concetto Base: Perché ci interessano così tanto "gli ordini di grandezza" invece del numero esatto di istruzioni di un codice? Perché calcolare il numero esatto (come 3217 passaggi) comporta enormi difficoltà matematiche e non fornisce reali vantaggi ingegneristici in prospettiva futura. Se hai un programma, l'unica cosa che conta per sapere se girerà su macchine scalabili, è avere uno "scontrino approssimativo" delle risorse in gioco. Dire che un codice è $O(n^2)$ è una stima del caso peggiore: ti garantisce che, decuplicando i dati, il tuo tempo aumenterà al più di circa 100 volte, non di un milione di volte.

Analisi Passo-Passo:

- L'O**-grande** è il tetto. Se hai un budget di tempo, ti assicura che il tempo del tuo algoritmo non sfonderà mai quel tetto (moltiplicato per una certa costante nascosta).
- L'Ω**-grande** è il pavimento. Ti dice il limite invalicabile.
- Il Θ**-grande** combina entrambi: stai dicendo che il tuo comportamento si incastra esattamente tra quel pavimento e quel tetto come in un tunnel asintotico. Se è $\Theta(n^2)$, crescerà esattamente in modo parabolico.

Il "Perché" è importante: La presenza di regole algebriche semplifica drammaticamente l'analisi del codice: se un tuo programma contiene due funzioni in sequenza, la prima con complessità $\Theta(n)$ e la seconda con $\Theta(n^3)$, grazie all'algebra asintotica sai già che l'intera procedura costerà globalmente $\Theta(n^3)$, perché il termine maggiore dominerà sempre "mangiandosi" tutti i termini minori non appena l'input diventerà sufficientemente grande.

Ripasso Visivo/Schematico:

Notazione	Intuizione Pratica	Vincolo Matematico Base
$O(g(n))$	Tetto massimo (Dominata)	L'algoritmo non fa MAI peggio di $g(n)$
$\Omega(g(n))$	Pavimento minimo (Dominante)	L'algoritmo non fa MAI meglio di $g(n)$
$\Theta(g(n))$	Risoluzione Stretta	Si comporta matematicamente in scala come $g(n)$

26/02/2026 Macchina RAM e i Criteri di Costo Computazionale

Output 1: Diario Generale di Ripasso (Formato Accademico)

- **Struttura di un problema:** Definizione di un modello di esecutore teorico privo di vincoli contingenti per poter misurare tempi operazionali oggettivi di un algoritmo, isolandolo dalle perturbazioni dell'hardware moderno.

- **Partiamo da una definizione formale:**
  - **Macchina RAM (Random Access Machine)**[5]
  - **Spazio degli Stati**[6]
  - **Criterio di Costo Uniforme (CCU)**[7]
  - **Criterio di Costo Logaritmico (CCL)**[8]
- **La soluzione:**
  - L'analisi matematica dell'esecutore RAM impone implicazioni fondamentali sulle risorse Spazio / Tempo:

    $$\text{Tempo Finito} \implies \text{Spazio Finito}$$

    (È impossibile fisicamente accedere a un'infinità di celle in un numero finito di passi esecutivi logici).
  - L'inversa è palesemente non valida (Spazio Finito $\not\Longleftarrow$ Tempo Finito). Con soli pochi registri la RAM può bloccarsi in cicli non terminanti.
  - In programmi con quadrature successive esponenziali, per esempio il calcolo di $z = 3^{2^n}$, l'istruzione elementare MUL sotto CCU valuta la prestazione in un fallace $O(n)$. Applicando l'aderente criterio CCL, il prodotto coinvolge stringhe di bit vertiginose, e la sommatoria dei loro pesi svela una reale natura esponenziale:

    $$T_P^l(n) = \Theta(2^n)$$

    Sancendo come un modello teoricamente più gravoso sia talvolta obbligatorio per mappare fedelmente i vincoli fisici.

Guida di Studio Approfondita e Semplificata (Formato Apprendimento)

Il Concetto Base: Perché serve la "Macchina RAM"? L'informatica ha bisogno di un arbitro imparziale. Se testassimo gli algoritmi con un cronometro, il risultato dipenderebbe da se usiamo un computer di oggi o un vecchio computer degli anni '80. La macchina RAM, invece, è un "computer di carta": esegue una sola operazione alla volta (leggere, sommare, spostare un dato), e noi contiamo quanti "colpi" le servono.

Analisi Passo-Passo: Abbiamo due modi distinti per dare un punteggio a questi "colpi":

- *Il Costo Uniforme (CCU):* È quello rapido e intuitivo. Fare una somma costa 1. Leggere un dato costa 1. Qualsiasi numero ci sia dentro, grande o piccolo, lo consideriamo di costo 1.

[5]**Macchina RAM (Random Access Machine):** Un modello computazionale astratto strutturato in 3 parti: programma sequenziale in memoria fissa, un accumulatore matematico-logico (R_0) unico che media tutte le operazioni, un nastro d'ingresso di sola lettura con termine delimitato dal simbolo *Blank* e un nastro d'uscita di sola scrittura.

[6]**Spazio degli Stati:** Il calcolo avviene tramite computazioni di passaggio di stato $\mathcal{S}$. Lo stato iniziale ha l'accumulatore, tutti gli infiniti registri R_k e il Program Counter lc azzerati. Le transizioni si evolvono tramite il ciclo Fetch-Decode-Execute.

[7]**Criterio di Costo Uniforme (CCU):** Un'operazione base costa rigidamente $\Theta(1)$ unità di tempo e ogni registro occupa $\Theta(1)$ in spazio indipendentemente dall'enormità del valore contenuto.

[8]**Criterio di Costo Logaritmico (CCL):** Il costo di operazione o di locazione è proporzionale al numero di bit dell'intero in memoria calcolato come $l(k) \approx \lfloor \log_2(k) \rfloor + 1$.

- *Il Costo Logaritmico (CCL):* A volte il primo metodo è un'illusione ottica. Se chiedo alla macchina di sommare numeri composti da due miliardi di cifre binarie, fare la somma in "1 secondo finto" è palesemente irreale. In questo caso entra in gioco il costo logaritmico, che dà un "pedaggio" a ogni operazione basato su quante cifre fisiche compongono quel numero (logaritmo in base 2).

Il "Perché" è importante: Scegliere il criterio sbagliato porta a catastrofi progettuali. Se hai un software classico (come ordinare vettori o cercare nomi), il Costo Uniforme è perfetto e molto snello da calcolare. Se stai creando algoritmi crittografici che calcolano radici e potenze enormi, usare il Costo Uniforme ti farebbe credere che il tuo algoritmo sia iperveloce, quando nella realtà la macchina impiegherebbe anni a scambiarsi i miliardi di bit di quei numeri (cosa correttamente prevista dal Costo Logaritmico).

Ripasso Visivo/Schematico:

Criterio	**Quando usarlo?**	**Vulnerabilità/Limitazioni**
Uniforme (CCU)	Programmi in cui la dimensione numerica del singolo dato rimane stabile e piccola (es. Liste, Vettori normali).	Ignora l'effettivo consumo hardware se un numero si espande all'infinito durante l'elaborazione.
Logaritmico (CCL)	Problematiche numeriche ad altissima precisione, calcoli esponenziali in modulo e quadrature.	Molto macchinoso da calcolare a causa delle serie matematiche da sviluppare ai logaritmi.

Capitolo 4

Modelli di Calcolo Teorici

02/03/2026 Architettura, Stato e Semantica della Macchina RAM

Output 1: Diario Generale di Ripasso (Formato Accademico)

- **Struttura di un problema:** Definizione formale dell'architettura, dello stato e delle regole di transizione di un esecutore astratto universale (la Macchina RAM) allo scopo di descrivere la semantica operazionale degli algoritmi.
- **Partiamo da una definizione formale:**
  - **Architettura RAM**[1]
  - **Stato della macchina (S)**[2]
  - **Stato Iniziale (S_0)**[3]
  - **Valore degli Operandi $V_S(op)$**[4]
- **La soluzione:**
  - L'esecuzione di un programma procede iterativamente attraverso il ciclo *Fetch-Decode-Execute*. La macchina individua l'istruzione tramite LC, decodifica l'operazione e l'operando e la esegue modificando lo stato S.
  - Per gestire cicli e diramazioni logiche si usano istruzioni di salto (es. `JUMP`, `JGTZ`, `JZERO`, `JBLANK`). Di norma LC si incrementa di 1, tranne in caso di salti attivati.
  - La semantica del linguaggio viene descritta definendo matematicamente la funzione calcolata da un programma P, mappando i vettori in ingresso $\mathbb{Z}^n$ su vettori di uscita.

$$\Phi_P(x) = \begin{cases} \text{vettore sul nastro di output} & \text{se la computazione si arresta in uno stato di HALT} \\ \perp \ \text{(Indeterminato)} & \text{se computazione infinita o operazione su operandi mal defi} \end{cases}$$

[1] **Architettura RAM:** Modello astratto composto da un nastro di ingresso (sola lettura), un nastro di uscita (sola scrittura), una memoria formata da una sequenza infinita di registri numerati R_k, un accumulatore R_0 (unico registro per le operazioni aritmetiche) e un Location Counter (LC) per le istruzioni.

[2] **Stato della macchina (S):** Una funzione formale $S : \{r, w, lc, 0, 1, \dots, k, \dots\} \to \mathbb{Z}$ che mappa le componenti hardware (posizione testina lettura r, scrittura w, Location Counter lc, registri) nei loro valori interi in un determinato istante (fotografia della macchina).

[3] **Stato Iniziale (S_0):** Configurazione d'avvio dove $S_0(r) = S_0(w) = S_0(lc) = 1$ e tutti i registri sono azzerati: $S_0(k) = 0, \forall k \in \mathbb{N}$. I dati di input sono pre-caricati sul nastro terminando con il simbolo speciale *Blank*.

[4] **Valore degli Operandi $V_S(op)$:** Funzione che valuta l'operando in base allo stato S. L'indirizzamento può essere: immediato ($= i \implies i$), diretto ($i \Rightarrow S(i)$), o indiretto ($*i \Rightarrow S(S(i))$ limitato a registri con indice $S(i) \geq 0$).

– Un classico caso applicativo visto a lezione è la ricerca del valore massimo in una sequenza. La macchina itera la lettura (`READ`), usa la sottrazione (`SUB`) e il test dell'accumulatore (`JGTZ`) per aggiornare il massimo parziale, fermandosi quando rileva il terminatore logico (`JBLANK`).

Guida di Studio Approfondita e Semplificata (Formato Apprendimento)

Il Concetto Base: Per studiare oggettivamente gli algoritmi non possiamo affidarci all'hardware di un computer moderno (troppo complesso e variabile nel tempo). Ci serve un "computer di carta": la Macchina RAM. È un modello semplicissimo ma capace di calcolare qualsiasi cosa un PC moderno possa calcolare. Ha un nastro da cui leggere infiniti dati (uno alla volta) e un nastro su cui stampare i risultati. La cosa essenziale è il concetto di "Stato": immaginate di premere il tasto pausa durante l'esecuzione del programma. Lo Stato è semplicemente l'elenco di tutti i numeri presenti nei registri e delle posizioni esatte in cui si trovano le testine in quel millisecondo.

Analisi Passo-Passo:

1. *Inizializzazione:* Accendendo la macchina, tutto si azzera (S_0). La testina di lettura è al blocco 1. Il programma (scritto in istruzioni elementari come ADD, SUB, JUMP) inizia dalla linea 1. Sul nastro di ingresso troviamo i dati del problema chiusi da un simbolo speciale (Blank), che fa da "muro" per indicare la fine.

2. *Indirizzamento della Memoria:* È fondamentale capire i tre modi in cui possiamo fornire un dato a un'istruzione:

   - **Immediato (=13):** Stiamo dicendo alla macchina "Usa esattamente il numero 13".
   - **Diretto (13):** Stiamo dicendo "Apri il cassetto (registro) numero 13 e usa il numero che ci trovi dentro".
   - **Indiretto (*13):** Stiamo dicendo "Apri il cassetto 13. Il numero che ci trovi dentro NON è il dato, ma è il nome di un ALTRO cassetto. Vai ad aprire quest'ultimo e usa il suo contenuto".

3. *Funzionamento del Codice:* L'algoritmo esegue un loop "Fetch-Decode-Execute" (pesca l'istruzione, capisci cosa fa, eseguila modificando lo Stato). Quando il programma finisce correttamente, sul nastro di uscita abbiamo la risposta. Se il programma si incarta o non finisce mai, dal punto di vista matematico l'output è "Indeterminato" (simbolo $\perp$).

Il "Perché" è importante: Creare un modello matematico come la macchina RAM trasforma un programma software in una funzione matematica pura. Questo è il pilastro fondante dell'informatica teorica: ci permette di studiare, contare e limitare oggettivamente il "costo" di un algoritmo in passi elementari, stabilendo teoremi che resisteranno all'evoluzione dei microprocessori reali.

Ripasso Visivo/Schematico:

Notazione	**Modalità**	**Formalizzazione Valore $V_S(op)$**
`=i`	Immediata	Il valore è l'intero i.
`i`	Diretta	Il valore è $S(i)$ (contenuto del registro i).
`*i`	Indiretta	Il valore è $S(S(i))$ (il contenuto del registro il cui indice è contenuto nel registro i).

Capitolo 5

Macchina RAM e Criteri di Costo Computazionale

03/03/2026 Macchina RAM: I/O Dinamico e Indirizzamento Indiretto

Output 1: Diario Generale di Ripasso (Formato Accademico)

- **Struttura di un problema:** Definizione di un paradigma per l'acquisizione di una sequenza di dati di input di cardinalità ignota a priori, trasferendoli dal nastro di ingresso alla memoria della Macchina RAM.
- **Partiamo da una definizione formale:**
  - **Terminatore Logico (BLANK)**[1]
  - **Vincolo di Allocazione**[2]
- **La soluzione:**
  - Utilizzo dell'indirizzamento indiretto sfruttando l'accumulatore (R_0) come puntatore dinamico all'indirizzo di destinazione.
  - Si inizializza l'accumulatore con l'indirizzo del primo registro disponibile (es. `LOAD =1`).
  - Il loop di lettura si articola nei seguenti passi logici:
    1. Verifica della condizione di terminazione: `JBLANK` verso l'etichetta di fine (o di elaborazione successiva).
    2. Lettura del dato in memoria tramite indirizzamento indiretto: `READ *0`. Il dato letto dalla cella corrente del nastro viene memorizzato nel registro il cui indice è attualmente contenuto in R_0.
    3. Incremento del puntatore: `ADD =1` sull'accumulatore per predisporlo alla scrittura nel registro successivo.
    4. `JUMP` all'inizio del ciclo.

[1] **Terminatore Logico (BLANK):** Simbolo speciale codificato sul nastro di ingresso per denotare la fine della sequenza dei dati validi.

[2] **Vincolo di Allocazione:** Non essendo noto il numero n di dati, è impossibile allocare staticamente un numero prefissato di registri. È necessario un approccio dinamico per mappare l'i-esimo dato nell'i-esimo registro.

– In uscita dal ciclo, i dati risultano allocati sequenzialmente nei registri da R_1 a R_n. L'accumulatore conterrà il valore $n + 1$.

Guida di Studio Approfondita e Semplificata (Formato Apprendimento)

Il Concetto Base: Immaginate di dover scaricare dei pacchi da un nastro trasportatore (nastro di input) e metterli in una fila infinita di cassetti (i registri della RAM). Il problema è che non sapete quanti pacchi arriveranno. Non potete dire a priori "riempi il cassetto 1, poi il 2, poi il 3", perché dovreste scrivere un programma infinito. La soluzione è usare un "bigliettino" (l'accumulatore R_0) su cui scrivete il numero del cassetto corrente. Leggete il pacco, guardate il bigliettino, mettete il pacco nel cassetto indicato, cancellate il numero sul bigliettino e scrivete quello successivo (+1). Vi fermate quando il nastro trasportatore è vuoto (simbolo BLANK).

Analisi Passo-Passo: L'istruzione chiave è `READ *0`. L'asterisco significa "indiretto". Se R_0 contiene il numero 5, la macchina non sovrascrive R_0 con il dato appena letto dal nastro, ma va ad aprire il cassetto numero 5 e ci infila il dato. Subito dopo, sommiamo 1 a R_0 (che diventa 6), pronti per il giro successivo. Questo semplice ciclo trasforma una sequenza esterna ignota in un array ordinato in memoria.

Il "Perché" è importante: Questa procedura illustra come un modello di calcolo teorico privo di strutture dati di alto livello (come array o liste allocabili dinamicamente) possa implementarle costruendo logicamente dei puntatori fisici. È la genesi dell'allocazione dinamica della memoria a basso livello.

Ripasso Visivo/Schematico:

Istruzione	**Stato di R_0 prima**	**Effetto**
`LOAD =1`	(Ignorato)	R_0 diventa 1. Inizia il ciclo.
`JBLANK Fine`	1	Se c'è BLANK, salta a Fine. Altrimenti continua.
`READ *0`	1	Legge il dato X dal nastro e lo copia in R_1.
`ADD =1`	1	Aggiunge 1 a R_0. Ora R_0 vale 2.
`JUMP Inizio`	2	Torna a `JBLANK`. Il prossimo dato finirà in R_2.

03/03/2026 Spazio, Tempo e Criteri di Costo (CCU vs CCL)

Output 1: Diario Generale di Ripasso (Formato Accademico)

- **Struttura di un problema:** Definizione di metriche oggettive e invarianti all'hardware per quantificare le risorse (Tempo e Spazio) richieste dall'esecuzione di un algoritmo sulla Macchina RAM. Analisi dell'asimmetria logica tra risorse e dei limiti del criterio uniforme.
- **Partiamo da una definizione formale:**
  - **Relazione Spazio/Tempo:** Tempo Finito $\implies$ Spazio Finito. L'implicazione inversa è rigorosamente falsa. *(Nota frammentaria: la dimostrazione empirica in aula mostra che è possibile creare un loop infinito operando su un singolo registro).*
  - **Criterio di Costo Uniforme (CCU)**[3]

[3] **Criterio di Costo Uniforme (CCU):** Modello in cui ogni operazione base e l'occupazione di ogni singolo registro hanno un costo rigido $\Theta(1)$, prescindendo dalla magnitudine dei valori elaborati.

- **Criterio di Costo Logaritmico (CCL)**[4]

- **La soluzione:**
  - L'utilizzo esclusivo del CCU può condurre a valutazioni fallaci qualora le assunzioni del modello non siano rispettate nella prassi (ovvero, se l'intero non è mappabile su un singolo registro fisico).
  - **Il paradosso computazionale delle quadrature successive:** Per calcolare $z = 3^{2^n}$, l'algoritmo esegue n moltiplicazioni in cascata ($y \leftarrow y \times y$).
  - Sotto **CCU**: il ciclo esegue una quantità costante di istruzioni n volte. Complessità temporale $T(n) = 8n + 7 = \Theta(n)$.
  - Sotto **CCL**: ad ogni passo k, il registro contiene il valore 3^{2^k}. La moltiplicazione `MULT 2` coinvolge un operando la cui lunghezza in bit scala come $\approx 2^k \log_2 3$. La sommatoria dei costi delle moltiplicazioni assume la forma:

    $$T_P^l(n) = \Theta\left(\sum_{k=0}^{n} 2^k\right) = \Theta(2^n)$$

    Passando dal CCU al CCL, la complessità subisce un'esplosione esponenziale.

Guida di Studio Approfondita e Semplificata (Formato Apprendimento)

Il Concetto Base: Perché diciamo che "Tempo Finito implica Spazio Finito"? Perché la macchina compie un'operazione per volta. Se il programma si spegne (finisce) dopo 1.000 operazioni, al massimo avrà potuto toccare 1.000 registri. Non è fisicamente possibile usare spazio infinito in tempo finito. Invece, "Spazio Finito NON implica Tempo Finito": un programma può bloccarsi in un loop usando sempre e solo il Registro 1 all'infinito. Lo spazio usato è minuscolo (1 registro), ma il tempo è infinito.

Analisi Passo-Passo: Valutare un algoritmo con il **Costo Uniforme (CCU)** significa far finta che sommare $1 + 1$ costi esattamente quanto sommare due numeri lunghi miliardi di cifre. È un'astrazione comodissima e funziona bene nel 99% delle applicazioni classiche (ordinare liste, cercare dati), perché di solito i numeri entrano comodamente nella memoria standard a 64-bit del computer. Tuttavia, quando si scrivono algoritmi che fanno esplodere la grandezza dei numeri (come in crittografia, calcolando potenze di potenze), questa "finta" crolla. Qui entra in gioco il **Costo Logaritmico (CCL)**: un'operazione costa in base a "quanto inchiostro serve per scrivere il numero in binario".

Il "Perché" è importante: L'esempio 3^{2^n} è illuminante. Il codice fa solo n moltiplicazioni (un loop brevissimo!). Nel mondo astratto (CCU), questo programma è velocissimo: cresce in modo lineare $\Theta(n)$. Ma fisicamente, ad ogni ciclo il numero raddoppia le proprie cifre. Il computer dovrà eseguire moltiplicazioni sempre più titaniche. Il costo reale per il silicio (CCL) tiene traccia di questo enorme sforzo di calcolo interno e ci rivela che l'algoritmo in realtà assorbe tempo esponenziale $\Theta(2^n)$. L'ingegnere che si affida ciecamente al costo uniforme per problemi numerici complessi sta progettando un sistema destinato a bloccarsi inesorabilmente.

Ripasso Visivo/Schematico:

Algoritmo: Calcolo 3^{2^n}	**Tempo Stimato (Matematica)**	**Classe di Crescita Asintotica**
Analisi con CCU	$8n + 7$ passi elementari	$\Theta(n)$ - Lineare (Illusionistica)
Analisi con CCL	$\sum_k (\approx 2^k$ bit)	$\Theta(2^n)$ - Esponenziale (Reale)

[4] **Criterio di Costo Logaritmico (CCL):** Modello che pondera il costo operazionale in funzione della rappresentazione binaria degli operandi. Il peso in bit di un intero k è $l(k) = \lfloor \log_2(|k|) \rfloor + 1$ se $k \neq 0$, e 1 se $k = 0$.

Capitolo 6

Limiti RAM, Macchina RASP e Calcolabilità

05/03/2026 Complessità Spaziale Logaritmica e Limiti della RAM

Output 1: Diario Generale di Ripasso (Formato Accademico)

- **Struttura di un problema:** Definizione formale della metrica per misurare il consumo effettivo di memoria (Spazio Logaritmico) durante l'intera esecuzione di un programma e analisi critica dei limiti del modello RAM classico.
- **Partiamo da una definizione formale:**
  - **Spazio Logaritmico** $S_P^l(x)$[1]
  - **Limiti del Modello RAM**[2]
  - **Precisione Notazionale (O vs Θ)**[3]
- **La soluzione:**
  - La formula che esprime la complessità in spazio logaritmico è la seguente:

$$S_P^l(x) = \max_{i \geq 0} \left(\sum_{j=0}^{\infty} l(S_i(j)) \right)$$

    dove $S_i(j)$ rappresenta il contenuto del registro j all'interno dello stato i-esimo della macchina, e $l(\cdot)$ è la funzione che ne calcola la lunghezza in bit.
  - Per algoritmi semplici (es. ricerca del massimo su input k), la complessità spaziale sotto costo uniforme è $\Theta(1)$ (es. 3 registri occupati). Sotto costo logaritmico, considerando la dimensione k dei dati, diventa $\Theta(\log k)$.

[1] **Spazio Logaritmico $S_P^l(x)$:** Il massimo spazio in bit occupato simultaneamente in memoria durante tutta la computazione. Si calcola massimizzando, su tutti gli stati $i \geq 0$, la somma delle lunghezze in bit dei dati contenuti in tutti i registri j.

[2] **Limiti del Modello RAM:** L'assunzione di spazio infinito e di registri capaci di contenere interi di dimensione illimitata è un'astrazione. È irrealistica per problemi con numeri che crescono esponenzialmente, ma accettabile se lo spazio non rappresenta il collo di bottiglia dell'applicazione.

[3] **Precisione Notazionale (O vs Θ):** L'utilizzo della notazione Θ fornisce un limite stringente (esatto a meno di costanti), mentre O-grande fornisce unicamente un limite superiore, risultando matematicamente corretto ma ingegneristicamente meno informativo se il limite è troppo lasco.

Guida di Studio Approfondita e Semplificata (Formato Apprendimento)

Il Concetto Base: Quando calcoliamo quanta memoria usa un programma con il Criterio Logaritmico, non basta contare "quanti cassetti" (registri) apriamo. Dobbiamo calcolare "quanto inchiostro" (bit) ci serve per scrivere i numeri in quei cassetti. Poiché i numeri cambiano durante l'esecuzione del programma (gli stati i), facciamo una foto alla memoria a ogni singolo passo, calcoliamo il totale dell'inchiostro usato in quella specifica foto, e alla fine della fiera consideriamo come costo totale il valore della foto in cui abbiamo consumato più inchiostro in assoluto (il massimo).

Analisi Passo-Passo: A lezione è stata sottolineata l'importanza di usare Θ al posto di O-grande quando possibile. È stato fatto un esempio molto chiaro: se so che il conto del ristorante è esattamente proporzionale a 40.000 euro (limite stretto, Θ), è un'informazione utile. Se dico solo che il conto sarà "minore o uguale a un miliardo di euro" (limite superiore, O-grande), sto dicendo una cosa matematicamente vera, ma totalmente inutile nella pratica! L'obiettivo dell'analisi è trovare stime che aderiscano il più possibile alla realtà del problema.

Il "Perché" è importante: Il modello RAM è fantastico perché è matematicamente semplice, ma i suoi limiti fisici ci impongono di usare i "criteri di costo" in base al contesto. Se calcolo il massimo in un vettore di temperature, i numeri sono piccoli e uso il costo uniforme (ignoro i limiti). Se faccio crittografia, i numeri esplodono: i registri illimitati non esistono nella realtà, quindi l'unico modo per avere stime corrette è misurare i bit con il costo logaritmico.

Ripasso Visivo/Schematico:

Notazione	**Esempio di dichiarazione**	**Valore Informativo**
$O(\log n)$	Il costo è *al massimo* logaritmico.	Basso (potrebbe costare molto meno, anche 1).
$\Theta(\log n)$	Il costo è *esattamente* logaritmico.	Alto (limite stretto, conosco il comportamento reale).

05/03/2026 Il Modello RASP e il Codice Automodificante

Output 1: Diario Generale di Ripasso (Formato Accademico)

- **Struttura di un problema:** Avvicinare l'esecutore astratto alle reali architetture hardware (macchine di Von Neumann), condividendo la memoria tra dati e istruzioni e simulando costrutti mancanti a livello hardware tramite manipolazione software.
- **Partiamo da una definizione formale:**
  - **Macchina RASP (Random Access Stored Program)**[4]
  - **Assenza di Indirizzamento Indiretto**[5]
  - **Codice Automodificante**[6]
- **La soluzione:**

[4] **Macchina RASP (Random Access Stored Program):** Modello computazionale equivalente alla RAM, in cui il programma risiede nella medesima memoria dei dati. Ad ogni istruzione sono associati due registri consecutivi: uno per il codice operativo (opcode) e uno per l'indirizzo.

[5] **Assenza di Indirizzamento Indiretto:** A differenza della RAM, la sintassi RASP non prevede nativamente l'indirizzamento indiretto (l'asterisco $*$), che deve essere simulato a livello logico.

[6] **Codice Automodificante:** Possibilità della macchina RASP di accedere ai registri che contengono le proprie istruzioni esecutive per sovrascriverne dinamicamente i campi operando prima che queste vengano eseguite dal *Location Counter* (LC).

- Per mantenere separati dati e programma in un'unica memoria, si definisce un indirizzo logico di "impianto" o "base" S da cui parte il blocco di istruzioni. I dati occuperanno registri in un'altra area.
- La simulazione dell'istruzione RAM `MULT *k` in RASP richiede circa 6 istruzioni.
- L'algoritmo calcola prima dinamicamente l'indirizzo fisico del dato (sommando opportuni offset R per traslare gli indici logici RAM in indici fisici RASP).
- Successivamente, utilizza un'istruzione `STORE` per inserire il valore calcolato esattamente nel registro che fungerà da "campo operando" per l'istruzione `MULT` che si trova poche righe più sotto nel codice. Quando l'LC raggiungerà la `MULT`, essa troverà l'operando già correttamente compilato.

Guida di Studio Approfondita e Semplificata (Formato Apprendimento)

Il Concetto Base: La macchina RAM ha un limite: tiene il programma in una zona magica separata dai dati (architettura Harvard). I nostri computer veri (architettura Von Neumann) mettono tutto insieme. La macchina RASP simula questa realtà: programma e dati convivono nella stessa infinita cassettiera. C'è però un dazio da pagare: la RASP non sa fare l'indirizzamento indiretto (il comando "vai ad aprire il cassetto il cui numero è scritto nel cassetto X").

Analisi Passo-Passo: Come si simula la mancanza del puntatore indiretto? Con un trucco incredibile: il programma modifica se stesso mentre gira!

1. Vogliamo moltiplicare usando un puntatore indiretto.
2. Usiamo 4-5 istruzioni per fare la matematica di base: guardiamo il cassetto X, leggiamo il numero (es. 100), e capiamo che il dato vero si trova nel cassetto 100.
3. A questo punto, il nostro programma "scrive" il numero 100 fisicamente dentro il campo della successiva istruzione di moltiplicazione.
4. Quando la macchina esegue la moltiplicazione, fa semplicemente MULT 100 come se fosse stato scritto così fin dall'inizio dal programmatore.

Il "Perché" è importante: Questo ci dimostra che avere istruzioni "comode" nell'hardware (come i puntatori diretti) non aumenta la potenza matematica di un computer. Tutto ciò che manca nell'hardware può essere simulato via software, a patto di accettare di dover eseguire qualche istruzione in più (overhead).

Ripasso Visivo/Schematico:

Caratteristica	**Macchina RAM**	**Macchina RASP**
Memoria Programma	Separata in sola lettura	Condivisa con i dati nei registri
Indirizzamento Indiretto	Presente nativamente (`*i`)	Assente, da simulare
Esecuzione	Statica (il codice non muta)	Dinamica (Codice automodificante)

05/03/2026 Equivalenza dei Modelli e Tesi di Church-Turing

Output 1: Diario Generale di Ripasso (Formato Accademico)

- **Struttura di un problema:** Dimostrare che la riduzione o l'aggiunta di funzionalità hardware non altera la classe dei problemi risolvibili (Calcolabilità Astratta), e definire l'impatto di tali variazioni sull'efficienza temporale (Calcolabilità Effettiva).
- **Partiamo da una definizione formale:**
  - **Teorema di Equivalenza RAM-RASP**[7]
  - **Tesi di Church-Turing**[8]
  - **Calcolabilità Effettiva (Classe P)**[9]
- **La soluzione:**
  - Tutti i linguaggi di programmazione completi (C, Java, Prolog, Macchina RAM, Macchina RASP) definiscono esattamente lo stesso set di funzioni calcolabili: $\mathcal{F}_{RAM} = \mathcal{F}_{RASP} = \mathcal{F}_{JAVA}$.
  - Quando si estende l'analisi al concetto di tempo (Tesi di Church estesa), la simulazione di un modello di calcolo su un altro modello aggiunge un onere (overhead) che può incrementare il grado del polinomio delle risorse temporali (es. da $\Theta(n^2)$ a $\Theta(n^4)$), ma un problema che si risolve in tempo polinomiale su una architettura rimarrà risolvibile in tempo polinomiale su qualsiasi altra architettura ragionevole.

Guida di Studio Approfondita e Semplificata (Formato Apprendimento)

Il Concetto Base: Esistono computer più potenti di altri da un punto di vista puramente matematico? La risposta è no. Che abbiate un supercomputer della NASA programmato in C++, o una noiosa macchina RASP di carta, entrambi possono risolvere esattamente gli stessi problemi. Questa è la Tesi di Church-Turing. L'unica differenza tra le due macchine sarà il tempo impiegato per arrivare alla soluzione.

Analisi Passo-Passo: Il teorema di equivalenza tra RAM e RASP ci dà un numero magico: 6. Visto che la RASP non ha i puntatori indiretti, deve "simularli" faticando di più. I matematici hanno dimostrato costruttivamente (creando un vero e proprio "compilatore" su carta) che nel caso peggiore in assoluto, un'istruzione della RAM viene tradotta in 6 istruzioni della RASP. Questo significa che la RASP è al massimo 6 volte più lenta della RAM. Un rallentamento lineare, un fattore costante che asintoticamente ignoriamo ($\Theta(n)$ rimane $\Theta(n)$).

Il "Perché" è importante: Da qui nasce il concetto di "Calcolabilità Effettiva" e dei Polinomi (Classe P). Il costo di tradurre e simulare un computer sopra un altro computer può appesantire i calcoli. Magari un algoritmo velocissimo $O(n^2)$ su un PC moderno diventa lento come $O(n^5)$ se lo fate simulare alla macchina RASP. Tuttavia, un polinomio resta un polinomio! Non diventerà mai un esponenziale $O(2^n)$. Per gli informatici, tutto ciò che è un polinomio è "praticamente calcolabile" (è efficiente), indipendentemente dal linguaggio che si sceglie per scriverlo.

Ripasso Visivo/Schematico:

[7]**Teorema di Equivalenza RAM-RASP:** Per ogni programma RAM esiste un programma RASP equivalente Ψ tale per cui, per ogni input x, la funzione calcolata è la medesima. Inoltre, il tempo di esecuzione è limitato asintoticamente: $T_{RASP}(x) \leq 6 \cdot T_{RAM}(x)$. La prova costruttiva trasforma ogni istruzione RAM in un blocco di massimo 6 istruzioni RASP.

[8]**Tesi di Church-Turing:** L'insieme delle funzioni calcolabili dalla macchina RAM (o RASP o Macchina di Turing) coincide con la classe delle funzioni ricorsive parziali. Questo limite delinea il concetto assoluto di "calcolabilità per via automatica".

[9]**Calcolabilità Effettiva (Classe P):** La classe dei problemi risolvibili su macchina RAM sotto Criterio Logaritmico in tempo limitato asintoticamente da un polinomio $\Theta(n^k)$. *(Nota: La dimostrazione formale dell'interprete RASP $\rightarrow$ RAM è trattata in modo frammentario nella registrazione, limitandosi al solo enunciato del teorema inverso).*

Concetto	**Cosa riguarda?**	**Significato Pratico**
Tesi Church-Turing	Calcolabilità Astratta	Se un problema non lo risolve la RAM, non lo risolverà mai neanche Java.
Tesi Church Estesa	Calcolabilità Effettiva	La traduzione tra linguaggi altera il tempo in modo polinomiale, non esponenziale.
Simulazione RASP	Efficienza di conversione	$T_{RASP} \leq 6 \cdot T_{RAM}$ (Overhead costante algebrico).

Capitolo 7

Linguaggi ad Alto Livello e Modularità

09/03/2026 Il Linguaggio AG: Variabili, Tipi e Semantica

Output 1: Diario Generale di Ripasso (Formato Accademico)

- **Struttura di un problema:** Astrarre i dettagli sintattici e hardware-dipendenti dei modelli RAM/RASP per concentrarsi esclusivamente sulla logica algoritmica, utilizzando un esecutore di alto livello.
- **Partiamo da una definizione formale:**
  - **Linguaggio AG (Algol Glossus)**[1]
  - **Left Value**[2]
  - **Right Value**[3]
  - **Puntatore**[4]
- **La soluzione:**
  - L'assegnamento di base $X = Y$ modifica il *Right Value* di X copiandovi il *Right Value* di Y.
  - L'utilizzo dell'operatore di deferenziazione (asterisco) altera la semantica dell'assegnamento.
  - Nell'assegnamento $*X = Z$, a sinistra dell'operatore c'è l'oggetto puntato. L'algoritmo non altera il *Right Value* del puntatore X, ma utilizza tale indirizzo per andare a sovrascrivere la variabile puntata con il *Right Value* di Z.
  - Nell'assegnamento $X = Y$ tra due puntatori, X assume come valore l'indirizzo contenuto in Y. Da quel momento, X e Y diventano *alias*, puntando fisicamente allo stesso oggetto in memoria.

[1] **Linguaggio AG (Algol Glossus):** Linguaggio di programmazione procedurale di alto livello utilizzato come pseudocodice. Supporta tipi di dato primitivi (interi, reali, stringhe) e strutturati (matrici, vettori, liste).

[2] **Left Value:** L'indirizzo di memoria fisico associato a una variabile (es. la cella di memoria all'indirizzo 1000).

[3] **Right Value:** Il contenuto effettivo o valore memorizzato all'interno dell'indirizzo associato alla variabile (es. il valore 7 contenuto all'indirizzo 1000).

[4] **Puntatore:** Una variabile il cui *Right Value* coincide con il *Left Value* (indirizzo) di un'altra variabile.

Guida di Studio Approfondita e Semplificata (Formato Apprendimento)

Il Concetto Base: Programmare direttamente con i registri della Macchina RAM è troppo dispersivo. Per studiare gli algoritmi ci serve un linguaggio più simile al nostro modo di ragionare, ma che sotto il cofano rispetti le regole matematiche della memoria. Il concetto più importante è separare il "contenitore" dal "contenuto". Il *Left Value* è l'etichetta sul cassetto (es. "Cassetto numero 1000"). Il *Right Value* è ciò che c'è dentro (es. "Il numero 7").

Analisi Passo-Passo: I puntatori spesso confondono, ma basta pensare ai *Right Value* e *Left Value*:

1. Se scrivo $X = Z$, sto dicendo: "Prendi il numero dentro Z e mettilo dentro il cassetto X".
2. Se X è un puntatore, il suo contenuto è il numero di un altro cassetto. Se scrivo $*X = Z$, l'asterisco dice: "Guarda dentro X, leggi il numero del cassetto bersaglio, vai a quel cassetto e mettici dentro il contenuto di Z".
3. Se ho due puntatori e scrivo $X = Y$, sto semplicemente copiando l'indirizzo. Risultato? Due frecce che indicano lo stesso identico oggetto.

Il "Perché" è importante: Comprendere questa differenza è il prerequisito assoluto per studiare le strutture dati dinamiche (come liste, alberi e grafi). Creare un alias ($X = Y$ tra puntatori) invece di copiare l'intera struttura dati salva enormi quantità di tempo e memoria, passando da una complessità lineare o quadratica a una complessità di tempo costante $\Theta(1)$.

Ripasso Visivo/Schematico:

Assegnamento	**Componente Modificata**	**Effetto Pratico**
$X = Z$	*Right Value* di X	Variabile semplice sovrascritta.
$*X = Z$	Oggetto puntato da X	La memoria all'indirizzo indicato da X viene sovrascritta. X non cambia puntamento.
$X = Y$ (puntatori)	*Right Value* di X	X e Y ora puntano allo stesso indirizzo di memoria (Alias).

09/03/2026 Strutture di Controllo e Sottoprogrammi

Output 1: Diario Generale di Ripasso (Formato Accademico)

- **Struttura di un problema:** Regolamentare il flusso di controllo logico (condizioni e iterazioni) e la modularizzazione del codice (procedure e funzioni), definendone in modo rigoroso l'impatto sulla complessità computazionale. *(Nota: la trattazione dei record di attivazione nello stack di memoria è qualitativa ed evocata a livello concettuale nelle registrazioni).*
- **Partiamo da una definizione formale:**
  - **Costrutto If-Then-Else**[5]
  - **Procedura vs Funzione**[6]

[5] **Costrutto If-Then-Else:** Operatore di diramazione logica. Valuta un'espressione booleana in tempo costante e indirizza il Program Counter verso il blocco C_1 o il blocco C_2.

[6] **Procedura vs Funzione:** Una funzione restituisce un valore tipizzato all'ambiente chiamante. Una procedura non restituisce valori, ma esiste esclusivamente per causare un *side-effect* (modifica dello stato globale della memoria o delle variabili passate per riferimento).

- **Passaggio per Valore (Copia)**[7]
- **Passaggio per Riferimento**[8]

- **La soluzione:**
  - **Calcolo della Complessità del costrutto If:** Poiché l'esecutore seguirà mutualmente o il ramo C_1 o il ramo C_2, il costo nell'analisi al caso peggiore è limitato superiormente dal massimo dei due costi:

$$T_{\text{if}} = \Theta(1) + \max(T(C_1), T(C_2))$$

  - **Impatto dei Sottoprogrammi (es. Swap):** Una procedura `Scambia(x, y)` necessita obbligatoriamente del passaggio per riferimento. Se venisse utilizzato il passaggio per valore, la procedura scambierebbe unicamente le copie locali nel proprio Stack, terminando la sua esecuzione senza indurre alcuna modifica sulle variabili originarie dell'ambiente chiamante.

Guida di Studio Approfondita e Semplificata (Formato Apprendimento)

Il Concetto Base: Quando il codice si dirama in un `If-Then-Else`, come calcoliamo il "conto" del nostro algoritmo? A lezione è stato usato un esempio perfetto: immaginate due amici al ristorante. Se si verifica la condizione X, il primo mangia una pizza. Se si verifica la condizione Y, il secondo mangia caviale. Non sapendo in anticipo quale condizione si verificherà, per essere sicuri di avere abbastanza soldi (analisi al caso peggiore), si porta con sé la cifra per il pasto più costoso. Il conto massimo dipenderà dal caviale, non dalla somma di pizza e caviale, perché solo uno dei due mangerà!

Analisi Passo-Passo: La differenza tra passaggio per *Valore* e *Riferimento* è vitale:

- *Passaggio per Valore:* È come dare a un amico la fotocopia di un tuo documento. Lui può scriverci sopra, strapparla o bruciarla. Il tuo documento originale a casa tua rimarrà intatto.
- *Passaggio per Riferimento:* È come dare all'amico le chiavi di casa tua per accedere al documento originale. Se lui lo modifica, quando tornerai a casa troverai il documento modificato.
- Se scriviamo una funzione `Scambia(A, B)`, dobbiamo passargli le chiavi (riferimento), altrimenti la funzione scambierà solo due fotocopie inutili e le butterà via appena finita.

Il "Perché" è importante: Scegliere come passare le variabili altera drasticamente la complessità spaziale e temporale. Se passi una matrice di 10.000 elementi per "valore", il computer perde tempo e memoria a creare 10.000 copie nel Record di Attivazione. Se la passi per "riferimento", il computer passa un singolo indirizzo di memoria: costo $\Theta(1)$.

Ripasso Visivo/Schematico:

Modalità Passaggio	Dato Trasferito	Conseguenze Pratiche
Per Valore	Copia del *Right Value*	Protegge i dati originali. Alto costo di copia per strutture dati grandi.
Per Riferimento	Indirizzo (*Left Value*)	Permette modifiche globali (side-effects). Costo di invio sempre costante $\Theta(1)$.

[7] **Passaggio per Valore (Copia):** Il *Right Value* del parametro attuale viene copiato nell'area di memoria (Record di Attivazione) del parametro formale. Modifiche locali non alterano la variabile originale.

[8] **Passaggio per Riferimento:** Viene passato il *Left Value* (indirizzo) del parametro attuale. Il parametro formale agisce come un alias; le modifiche alterano lo stato della variabile chiamante.

Capitolo 8

Strutture Dati: Dalla Memoria Statica all'Allocazione Dinamica

10 Marzo 2026 Mappe di Memorizzazione e Ottimizzazione degli Indici

Output 1: Diario Generale di Ripasso (Formato Accademico)
Struttura di un problema:

- **Partiamo da una definizione formale: Mappa di Memorizzazione**[1]
- **Memorizzazione per Righe (Row-Major)**[2]
- **La soluzione:** Il costo temporale per accedere a un elemento deve essere $\Theta(1)$. L'equazione di indirizzamento fisico per una matrice $M \times N$ con indici che partono da 1 ($1 \leq i \leq M, 1 \leq j \leq N$) richiede il salto delle prime $i-1$ righe e dei primi $j-1$ elementi sulla riga corrente:

$$Indirizzo(i,j) = \alpha + k \cdot \Big[(i-1) \cdot N + (j-1)\Big]$$

Dove α è l'indirizzo base e k è la dimensione in byte del tipo base. L'ottimizzazione architetturale impone l'uso di indici a partire da 0 (zero-based indexing), semplificando l'equazione di ricorrenza interna del calcolo a:

$$Indirizzo(i,j) = \alpha + k \cdot (i \cdot N + j)$$

Questa trasformazione riduce il numero di operazioni aritmetiche atomiche da 7 (3 moltiplicazioni, 2 somme, 2 sottrazioni) a 5 (3 moltiplicazioni, 2 somme).

Guida di Studio Approfondita

Il Concetto Base: La RAM del computer è un unico lungo "nastro" di cassetti. Una matrice (es. 10×10) è un oggetto 2D logico che deve essere "appiattito" per entrarci. Se cerchi l'elemento

[1] **Mappa di Memorizzazione:** Funzione matematica utilizzata dal compilatore per linearizzare una struttura dati multidimensionale in una memoria monodimensionale (RAM), traducendo indici logici in indirizzi fisici assoluti.

[2] **Memorizzazione per Righe (Row-Major):** Criterio di linearizzazione per matrici di ordine $M \times N$ in cui gli elementi della prima riga precedono quelli della seconda, e così via.

alla riga 5, colonna 3, il computer non scorre i cassetti uno a uno: calcola esattamente l'indirizzo matematico saltando le prime 4 righe intere. Questo garantisce che trovare il 1° o il 1000° elemento richieda sempre lo stesso identico tempo asintotico.

Analisi Passo-Passo (Perché gli array partono da 0): Nei linguaggi moderni (come C o Java) si parte da 0 per pura efficienza hardware.

1. Se parti da 1, la CPU deve calcolare $i - 1$ e $j - 1$ ogni singola volta che apri le parentesi quadre della matrice `M[i][j]`.

2. Se parti da 0, i "-1" scompaiono dalla formula. Risparmiare 2 sottrazioni per ogni accesso a una matrice, ripetuto miliardi di volte in un ciclo `for`, trasforma un calcolo teorico in un guadagno prestazionale netto (dal 33% di operazioni in meno).

Il "Perché" è importante: Capire come i dati vengono disposti in memoria (per righe o per colonne) è cruciale per la località spaziale delle cache moderne. Ma soprattutto, giustifica scelte sintattiche storiche dei linguaggi di programmazione, derivate da rigidi vincoli di ottimizzazione del ciclo Fetch-Decode-Execute.

Ripasso Visivo/Schematico:

Parametro	**Indici 1-based (es. Algol, Pascal)**	**Indici 0-based (es. C, Java)**
Formula Row-Major	$\alpha + k \cdot [(i-1)N + (j-1)]$	$\alpha + k \cdot (i \cdot N + j)$
Costo Calcolo RAM	7 operazioni aritmetiche	5 operazioni aritmetiche
Tempo d'Accesso	$\Theta(1)$	$\Theta(1)$ (più veloce in costante)

Capitolo 9

Strutture Dati Elementari e Mappe di Memorizzazione

10/03/2026 Semantica dei Puntatori e Assegnamento

Output 1: Diario Generale di Ripasso (Formato Accademico)

- **Struttura di un problema:** Definizione formale della semantica delle operazioni di assegnamento nel linguaggio di alto livello AG, con particolare riferimento all'operatore di deferenziazione su variabili di tipo puntatore.
- **Partiamo da una definizione formale:**
  - **Variabile Puntatore**[1]
  - **Operatore di Deferenziazione ($*$)**[2]
  - **Alias**[3]
- **La soluzione:**
  - Nell'istruzione di assegnamento, il comportamento differisce radicalmente in base alla presenza dell'operatore di deferenziazione a sinistra o a destra del simbolo di uguaglianza.
  - Caso 1: $*X = Z$. A destra l'esecutore preleva il Right Value di Z. A sinistra, l'operatore indica l'oggetto puntato. L'algoritmo non altera il puntatore X, ma va a sovrascrivere l'area di memoria puntata da X con il valore di Z.
  - Caso 2: $*X = *Y$. A destra si estrae il Right Value dell'oggetto puntato da Y. Tale valore viene copiato nell'oggetto puntato da X.
  - Caso 3: $X = Y$. Entrambe le variabili sono puntatori senza deferenziazione. L'esecutore copia l'indirizzo contenuto in Y dentro X. Si crea un alias: X e Y ora puntano allo stesso identico oggetto in memoria. L'operazione ha costo costante:

$$T(X = Y) = \Theta(1)$$

[1] **Variabile Puntatore:** Una variabile il cui Right Value rappresenta l'indirizzo di memoria (Left Value) di un'altra variabile.

[2] **Operatore di Deferenziazione ($*$):** Operatore che, applicato a un puntatore X (es. $*X$), seleziona l'oggetto puntato, ovvero la variabile allocata all'indirizzo contenuto nel Right Value di X.

[3] **Alias:** Condizione in cui due variabili puntatore distinte contengono il medesimo Right Value (indirizzo), puntando conseguentemente alla medesima area di memoria fisica.

Guida di Studio Approfondita e Semplificata (Formato Apprendimento)

Il Concetto Base: Per non fare confusione con i puntatori, bisogna sempre distinguere tra la "freccia" (il puntatore) e il "bersaglio" (l'oggetto puntato). Se scrivo X, sto parlando della freccia. Se scrivo $*X$, sto parlando del bersaglio colpito dalla freccia. Quando leggiamo un'istruzione di assegnamento, dobbiamo guardare attentamente dove si trova l'asterisco.

Analisi Passo-Passo: Immaginiamo che X e Y puntino a due libri diversi (strutture dati grandi).

1. Se scrivo $*X = *Y$, sto dicendo: "Prendi il contenuto del libro puntato da Y, copialo parola per parola, e incollalo sopra il libro puntato da X". Questa è un'operazione lenta (costo proporzionale alla grandezza del libro).

2. Se scrivo $X = Y$ (senza asterischi), sto dicendo: "Prendi la freccia X e spostala in modo che punti allo stesso libro a cui punta Y". Il vecchio libro di X viene abbandonato (e poi rimosso dal Garbage Collector), e ora abbiamo due frecce che indicano un unico libro. Questa operazione è istantanea.

Il "Perché" è importante: La differenza tra queste operazioni è il cuore dell'efficienza algoritmica sulle strutture dati dinamiche. Copiare intere matrici o record costa tempo e spreca memoria. Creare alias ($X = Y$) richiede un tempo costante ed è il meccanismo fondamentale per collegare i nodi in liste, alberi e grafi.

Ripasso Visivo/Schematico:

Istruzione	Significato	Costo Computazionale
$*X = *Y$	Copia del contenuto della variabile puntata.	Proporzionale alla dimensione del dato copiato.
$*X = Z$	Sovrascrittura dell'oggetto puntato con una variabile.	Proporzionale alla dimensione di Z.
$X = Y$	Copia dell'indirizzo (creazione di un Alias).	Sempre $\Theta(1)$ (costante).

10/03/2026 Vettori: Definizione e Mappe di Memorizzazione

Output 1: Diario Generale di Ripasso (Formato Accademico)

- **Struttura di un problema:** Rappresentare in memoria centrale ad accesso casuale una collezione omogenea e lineare di dati (struttura dati Vettore), garantendo un tempo di accesso deterministico e costante per ogni elemento.

- **Partiamo da una definizione formale:**
  - **Vettore**[4]
  - **Proiezione (π)**[5]
  - **Sostituzione (σ)**[6]

[4]**Vettore:** Fissato un tipo base U e un intero n, un vettore di lunghezza n è un elemento del prodotto cartesiano $U^n = U \times U \times \cdots \times U$.

[5]**Proiezione (π):** Operazione di accesso in lettura. Estrae l'elemento a_i dato l'indice i. Se i non è compreso tra 1 e n, restituisce indefinito ($\perp$). Corrisponde all'istruzione `x = A[i]`.

[6]**Sostituzione (σ):** Operazione di accesso in scrittura. Genera un nuovo vettore in cui l'elemento in posizione i viene sovrascritto con un nuovo valore a. Corrisponde all'istruzione `A[i] = z`.

- **La soluzione:**
  - Per implementare questa struttura logica su una memoria fisica lineare (Macchina RAM), i dati vengono allocati in locazioni consecutive.
  - Siano: α l'indirizzo base del vettore, k la dimensione in byte/registri occupata dal tipo base U, e i l'indice cercato.
  - **Mappa con indici partenti da 1:** Per raggiungere l'elemento i, occorre "saltare" i precedenti $i-1$ elementi. La formula dell'indirizzo di memoria è:

$$Indirizzo = \alpha + (i-1) \cdot k$$

  - **Mappa con indici partenti da 0 (Ottimizzazione C/Java):** Se il primo elemento ha indice 0, l'elemento di indice i è preceduto esattamente da i elementi. La formula si semplifica:

$$Indirizzo = \alpha + i \cdot k$$

    Questa scelta risparmia un'operazione di sottrazione a basso livello (passando da 3 operazioni a 2), garantendo sempre un tempo di accesso $\Theta(1)$.

Guida di Studio Approfondita e Semplificata (Formato Apprendimento)

Il Concetto Base: Un vettore (array) è come una fila di cassette postali tutte uguali. In informatica, le operazioni fondamentali sono due: "guardare cosa c'è dentro una cassetta" (Proiezione) e "mettere un nuovo oggetto nella cassetta" (Sostituzione). Il computer deve poter trovare la cassetta numero 50 istantaneamente, senza dover contare partendo dalla prima.

Analisi Passo-Passo: Come fa il computer a calcolare l'indirizzo esatto al volo? Usa una formula chiamata "mappa di memorizzazione".

1. Immaginate che la prima cassetta si trovi al metro 1000 (α) e che ogni cassetta sia larga 5 metri (k).
2. Se le cassette sono numerate da 1 in poi, per trovare la cassetta 3, il computer calcola: 1000 + (3-1) * 5 = 1010. Deve sottrarre 1.
3. Se le cassette partono da 0 (come nei linguaggi moderni Java e C), la cassetta "3" in realtà è la quarta. Per trovarla, il calcolo è diretto: 1000 + (3 * 5) = 1015. Meno matematica da fare per il processore!

Il "Perché" è importante: Il motivo storico e ingegneristico per cui in quasi tutti i linguaggi di programmazione moderni gli indici dei vettori partono da zero non è una stranezza dei programmatori, ma una precisa scelta di ottimizzazione hardware: eliminare una sottrazione ad ogni singolo accesso alla memoria aumenta l'efficienza globale dell'esecuzione.

Ripasso Visivo/Schematico:

Base Indici	Formula Indirizzo	Operazioni Macchina
Indici 1-based	$\alpha + (i-1) \cdot k$	1 Sottrazione, 1 Moltiplicazione, 1 Somma
Indici 0-based	$\alpha + i \cdot k$	1 Moltiplicazione, 1 Somma (Più veloce)

10/03/2026 Matrici e Record: Strutture Dati Multidimensionali ed Eterogenee

Output 1: Diario Generale di Ripasso (Formato Accademico)

- **Struttura di un problema:** Mappare strutture dati complesse bidimensionali (Matrici) o composte da campi non omogenei (Record) su un'architettura di memoria che fisicamente ammette una sola dimensione lineare. *(Nota: la parte audio relativa alla derivazione empirica dei campi dei record presenta lievi divagazioni discorsive, ma il concetto formale resta rigoroso).*
- **Partiamo da una definizione formale:**
  - **Matrice**[7]
  - **Record**[8]
- **La soluzione:**
  - **Mappa per Matrici (Memorizzazione per Righe):** La matrice viene appiattita salvando prima tutta la riga 1, poi tutta la riga 2, ecc. L'indirizzo fisico dell'elemento di coordinate (i, j) per indici a base 1 si calcola saltando prima le righe intere e poi gli elementi nella riga corrente:

    $$Indirizzo = \alpha + (i-1) \cdot n \cdot k + (j-1) \cdot k$$

    Ottimizzazione per indici a base 0:

    $$Indirizzo = \alpha + i \cdot n \cdot k + j \cdot k$$

    Il costo di accesso permane limitato asintoticamente a $\Theta(1)$.
  - **Mappa per Record:** Poiché i campi U_i hanno dimensioni k_i eterogenee, non è possibile usare una formula moltiplicativa generica. Tuttavia, l'etichetta del campo è esplicitata nel codice. L'indirizzo del campo e_i è dato da:

    $$Indirizzo = \alpha + \sum_{j=1}^{i-1} k_j$$

    Poiché l'etichetta è nota staticamente a tempo di compilazione, il compilatore risolve la sommatoria degli offset in anticipo (calcolo pre-assemblato). Il tempo di accesso a runtime per il programma è puro $\Theta(1)$.

Guida di Studio Approfondita e Semplificata (Formato Apprendimento)

Il Concetto Base: La RAM del computer è come un lunghissimo e sottile nastro di carta. Non ci sono griglie bidimensionali nella memoria fisica! Quindi, come salviamo una griglia a scacchiera (matrice)? La tagliamo riga per riga e incolliamo le righe una di seguito all'altra sul nostro nastro. Per quanto riguarda i Record (pensate a una scheda anagrafica: il Nome è una stringa lunga, l'Età è un numeretto corto), abbiamo scompartimenti di dimensioni tutte diverse attaccati in sequenza.

Analisi Passo-Passo:

[7]**Matrice:** Fissato un tipo base U e due interi m, n, una matrice di ordine $m \times n$ è un elemento dell'insieme $U^{m \times n}$. Richiede due indici (i, j) per la proiezione e sostituzione.

[8]**Record:** Un contenitore di informazione non uniforme costituito da n campi. Formalmente appartiene al prodotto cartesiano di n tipi possibilmente diversi: $U_1 \times U_2 \times \cdots \times U_n$.

- **Per la Matrice:** Se vogliamo l'elemento in riga 4 e colonna 7, il computer salta fisicamente le prime 3 righe complete (offset delle righe), "atterra" all'inizio della quarta riga e fa un ulteriore salto di 6 posizioni (offset della colonna). È tutta matematica lineare.

- **Per il Record:** Visto che i campi sono di misure diverse, non c'è una tabellina matematica universale. Ma c'è un trucco: il programmatore scrive 'Persona.Età'. Il compilatore sa già che prima dell''Età' c'è il 'Nome' (che occupa, per esempio, 20 byte). Quindi traduce tutto in: "Vai all'indirizzo base + 20 byte".

Il "Perché" è importante: La linearizzazione della memoria è la vera magia che permette ai nostri programmi di gestire concetti 2D, 3D o astrazioni eterogenee come oggetti e struct, pur mantenendo sempre l'assoluta garanzia di poter prelevare qualsiasi informazione in un tempo immediato e costante ($\Theta(1)$), indipendentemente da quanto sia grande la matrice o il database.

Ripasso Visivo/Schematico:

Struttura	**Logica di Mappatura**	**Calcolo dell'Indirizzo**
Matrice (0-based)	Scansione per righe intere e poi per colonne.	Matematico a runtime: $\alpha + i \cdot n \cdot k + j \cdot k$
Record	Sequenza di byte di dimensioni arbitrarie per ogni campo.	Somma degli offset calcolata in anticipo dal compilatore.

Capitolo 10

Strutture Dati Dinamiche: Liste e Accesso Ristretto

12/03/2026 Struttura Dati Lista, Implementazione e Memoria

Output 1: Diario Generale di Ripasso (Formato Accademico)

- **Struttura di un problema:** Superare il vincolo dimensionale rigido dei vettori allocati staticamente, definendo una struttura dati lineare capace di espandersi o contrarsi dinamicamente nel corso dell'esecuzione.
- **Partiamo da una definizione formale:**
  - **Lista**[1]
  - **Lettura (Proiezione)**[2]
  - **Inserimento e Cancellazione**[3]
  - **Nodo**[4]
- **La soluzione:**
  - **Implementazione tramite Vettori Paralleli:** Simulazione logica dei puntatori usando due vettori (es. `Nome` e `Succ`). Il vettore `Succ` ospita gli indici logici del prossimo elemento, con il valore 0 o un simbolo speciale utilizzato come terminatore (*nil*).
  - **Implementazione con Puntatori Fisici:** Sfruttamento della memoria dinamica (Heap). L'inserimento in testa avviene in tempo puramente costante tramite manipolazione dei riferimenti:

$$T_{\text{InsTesta}} = \Theta(1)$$

[1]**Lista:** Fissato un tipo base U, una lista è un elemento dell'unione infinita $\bigcup_{i \geq 0} U^i$. È una sequenza logica di elementi in cui ciascuno è associato a una specifica posizione (1° elemento, 2° elemento, ecc.).

[2]**Lettura (Proiezione):** Restituisce l'elemento in posizione k. Se k eccede la lunghezza della lista, l'operazione restituisce il valore indefinito ($\perp$).

[3]**Inserimento e Cancellazione:** Operazioni dinamiche che modificano la lunghezza della lista. L'inserimento in posizione k (con $1 \leq k \leq n+1$) slitta gli elementi successivi. La cancellazione elimina l'elemento in k ricompattando la sequenza.

[4]**Nodo:** Record composto da due campi: un campo dati (*elem*) e un campo puntatore (*succ* o *next*) che contiene il *Left Value* del nodo successivo.

- **Costo di Accesso Posizionale:** A differenza dei vettori, per accedere al k-esimo elemento di una lista non esiste una mappa di memorizzazione aritmetica. L'esecutore è obbligato ad attraversare sequenzialmente i primi $k-1$ nodi, con un costo di accesso lineare $\Theta(n)$.
- **Gestione della Memoria (Freelist):** *(Nota: Il meccanismo di rilascio e richiesta memoria è descritto in modo parzialmente qualitativo e frammentario nell'audio).* L'allocazione di un nodo preleva il primo blocco libero dalla cosiddetta *freelist* aggiornando il puntatore di testa in tempo $\Theta(1)$. La deallocazione restituisce il blocco ponendolo come nuova testa della *freelist*, anch'essa in $\Theta(1)$.

Guida di Studio Approfondita e Semplificata (Formato Apprendimento)

Il Concetto Base: Pensate a un vettore come a un casellario postale fissato nel muro: trovate subito lo scomparto 50, ma se volete aggiungere un nuovo scomparto in mezzo, dovete spaccare il muro. La Lista, invece, è come una caccia al tesoro. Avete un indizio iniziale in mano (la testa della lista) che vi porta al primo nascondiglio. Lì trovate l'informazione che cercavate e un secondo biglietto con l'indirizzo del nascondiglio successivo. Non sapete in anticipo dove si trova l'elemento 50; dovete obbligatoriamente seguire tutti i 49 indizi precedenti per arrivarci.

Analisi Passo-Passo: Quando aggiungiamo un elemento all'inizio della lista (Inserimento in Testa), compiamo tre azioni meccaniche velocissime:

1. Creiamo un nuovo "nascondiglio" vuoto.
2. Mettiamo dentro questo nascondiglio l'indirizzo della vecchia testa della lista.
3. Aggiorniamo il nostro punto di partenza (il primo indizio in nostro possesso) facendolo puntare a questo nuovo nascondiglio appena creato.

Tutto questo richiede 3 operazioni, ignorando completamente quanto sia lunga la coda della lista.

Il "Perché" è importante: La lista sacrifica la velocità di accesso ai dati centrali per premiare la dinamicità. È la struttura dati principe quando non si conosce a priori la quantità di dati da memorizzare, evitando così memory overflow o sprechi di memoria pre-allocata inutilizzata.

Ripasso Visivo/Schematico:

Struttura	**Costo Accesso i-esimo**	**Costo Inserimento Testa**	**Dimensione**
Vettore	$\Theta(1)$ Costante	$\Theta(n)$ Lineare (slittamento)	Statica
Lista	$\Theta(n)$ Lineare (scorrimento)	$\Theta(1)$ Costante	Dinamica

12/03/2026 Liste Circolari e il Problema di Giuseppe Flavio

Output 1: Diario Generale di Ripasso (Formato Accademico)

- **Struttura di un problema:** Simulare un processo ciclico di eliminazione basato su un conteggio modulare, dimostrando l'impatto critico della scelta della struttura dati sulla complessità computazionale globale dell'algoritmo.
- **Partiamo da una definizione formale:**

- **Problema di Giuseppe Flavio**[5]
- **Lista Circolare**[6]

- **La soluzione:**
  - **Approccio con Lista Circolare:** Si scorre la lista avanzando di $M-1$ passi. Arrivati al nodo precedente a quello da eliminare, si modifica il suo puntatore per "scavalcare" il nodo vittima, collegandolo al nodo successivo: `(X*).succ := (*(*X).succ).succ` L'operazione di by-pass (cancellazione) avviene in tempo $\Theta(1)$. Contare per M passi costa $\Theta(M)$. Ripetendo per $N-1$ individui da eliminare, il costo asintotico è $\Theta(N \cdot M)$.
  - **Criticità dell'approccio Vettoriale:** Se si simulasse l'anello in un array ponendo marcatori logici (es. "morto" = -1), la scansione per trovare il prossimo superstite diventerebbe via via più onerosa. Per passare da un elemento all'altro occorrerebbe attraversare iterativamente i "buchi" lasciati dagli elementi già rimossi. Nella fase terminale del gioco, questo approccio degenera rapidamente portando la complessità computazionale verso un andamento strettamente quadratico:

$$T_{vettore}(N) = \Theta(N^2)$$

  - Conclusione: Per $M \ll N$ (piccole costanti), l'approccio con lista circolare fornisce un tempo $\Theta(N)$ asintoticamente ottimale rispetto al pessimo $\Theta(N^2)$ del vettore.

Guida di Studio Approfondita e Semplificata (Formato Apprendimento)

Il Concetto Base: Il problema storico di Giuseppe Flavio (i ribelli che si auto-eliminano in cerchio saltando di 3 in 3) è il perfetto test di stress per le strutture dati. Come programmiamo un computer per simulare questo cerchio mortale? Possiamo usare una *Lista Circolare*, dove semplicemente l'ultimo nodo ha una freccia che si ricollega al primo. Quando qualcuno esce dal cerchio, chi gli stava alla destra prende la mano di chi gli stava alla sinistra, e il cerchio si restringe fisicamente.

Analisi Passo-Passo: Perché l'array in questo caso è una pessima idea? In un array, i posti sono fissi. Se eliminiamo la persona al posto 3, il posto 3 rimane vuoto. Facendo questo gioco fino a che restano in vita poche persone su 1000, il nostro codice dovrà faticosamente attraversare centinaia di posti vuoti (i "fantasmi") per riuscire a trovare la prossima persona in vita da contare. Più il gioco avanza, più il computer perde tempo a contare posti vuoti, trasformando un problema lineare in un lentissimo calcolo quadratico. La lista concatenata, invece, "scollega" davvero l'elemento eliminato, rimpicciolendo i tempi di percorrenza passo dopo passo.

Il "Perché" è importante: Questo problema dimostra che non basta sapere scrivere un ciclo o conoscere l'algoritmo corretto. Applicare la logica corretta sopra la struttura dati sbagliata (l'array invece della lista circolare) porta a un tracollo inesorabile delle prestazioni software.

Ripasso Visivo/Schematico:

Contenitore	Gestione dell'eliminato	Costo di Esecuzione
Lista Circolare	By-pass dei puntatori, il nodo "svanisce" dal cammino.	$\Theta(N \cdot M)$ (Spesso Lineare)
Vettore Simula- to Circolarmente	Marcatura logica (es. $= 0$). Costringe l'attraversamento a vuoto.	$\Theta(N^2)$ (Quadratico e lento)

[5]**Problema di Giuseppe Flavio:** Dati N individui disposti in cerchio e un passo di conteggio M, si elimina iterativamente l'individuo in posizione M-esima a partire dall'ultimo eliminato. Determinare l'indice dell'unico superstite.

[6]**Lista Circolare:** Variante topologica della lista concatenata in cui il campo puntatore dell'ultimo elemento non è nullo (*nil*), ma punta fisicamente al primo elemento della lista, creando un anello ininterrotto.

Output 1: Diario Generale di Ripasso (Formato Accademico)

- **Struttura di un problema:** Regolamentare l'ordine logico di accesso, inserimento ed estrazione da una collezione di dati imponendo discipline rigide di servizio per supportare particolari esigenze algoritmiche o di sistema operativo.
- **Partiamo da una definizione formale:**
  - **Pila (Stack)**[7]
  - **Operazioni Pila**[8]
  - **Coda (Queue)**[9]
  - **Operazioni Coda**[10]
- **La soluzione:**
  - **Implementazione tramite Liste Concatenate:**
    1. *Pila:* L'estremità *Top* viene fatta coincidere logicamente con la testa della lista concatenata. Un inserimento `Push` è un inserimento in testa; un'estrazione `Pop` è una cancellazione in testa. Costo temporale garantito per tutte le operazioni: $\Theta(1)$.
    2. *Coda:* Richiede il mantenimento simultaneo di due puntatori ausiliari, `First` (punta al primo nodo in uscita) e `Last` (punta all'ultimo nodo per inserimenti). Una `Enqueue` aggiorna il `Next` dell'elemento `Last` e fa avanzare il puntatore. Anche qui l'uso della lista evita buffer overflows non essendoci capienze prefissate. Costo operazioni: $\Theta(1)$.

Guida di Studio Approfondita e Semplificata (Formato Apprendimento)

Il Concetto Base: Mentre in una lista normale avete il permesso di guardare, inserire o cancellare dati ovunque, le Pile e le Code vi tagliano le mani, obbligandovi a regole di comportamento ben precise.

- La **Pila (Stack)** è come una pila di piatti da lavare: l'ultimo piatto che appoggiate in cima, sarà il primo che prenderete per lavarlo (LIFO).
- La **Coda (Queue)** è letteralmente la fila all'ufficio postale. L'ultimo che entra in ufficio, si mette in fondo. Il primo che verrà servito dallo sportello è quello che era entrato prima di tutti ed è in testa alla fila (FIFO).

Analisi Passo-Passo: Implementare queste strutture usando liste con i puntatori è geniale per la sua velocità:

[7] **Pila (Stack):** Struttura dati lineare governata dalla disciplina *LIFO (Last In, First Out)*. Le operazioni di lettura, modifica e inserimento avvengono esclusivamente a una singola estremità, detta *Top* (Cima).

[8] **Operazioni Pila:** `Push` (inserisce in cima), `Pop` (rimuove l'elemento in cima), `Top` (restituisce in sola lettura il valore in cima), `IsEmpty` (verifica condizione di vuoto).

[9] **Coda (Queue):** Struttura dati lineare governata dalla disciplina *FIFO (First In, First Out)*. Gli elementi entrano in una estremità (*Last/Rear*) ed escono dall'estremità opposta (*First/Front*).

[10] **Operazioni Coda:** `Enqueue` (inserisce in coda), `Dequeue` (rimuove dalla testa), `Front` (restituisce il valore del primo elemento servibile).

1. Nella *Pila*, ci basta avere in mano solo il puntatore alla cima. Se voglio togliere, taglio la testa ($\Theta(1)$). Se voglio aggiungere, incollo un nuovo elemento in testa ($\Theta(1)$).

2. Nella *Coda*, ci servono due mani: una mano fissa al primo della fila per estrarlo quando servito (first), e l'altra fissa all'ultima persona della fila per poter agganciare i nuovi arrivati (last). Operare su questi due capi è istantaneo, ancora una volta $\Theta(1)$.

Il "Perché" è importante: La Pila è il cuore del funzionamento interno del computer (come la gestione delle chiamate a funzione e del Record di Attivazione). La Coda è la struttura vitale nei Sistemi Operativi per gestire chi deve usare le risorse condivise (come i lavori inviati alla stampante o le priorità del processore). Senza queste due strutture ad accesso rigidamente controllato, regnerebbe il caos algoritmico.

Ripasso Visivo/Schematico:

Struttura	**Logica**	**Estremità Utilizzate**	**Complessità Operazioni**
Pila (Stack)	LIFO	Solo la Testa (Top).	Inserimenti/Estrazioni $\Theta(1)$
Coda (Queue)	FIFO	Inserimento in Fondo (Last), Estrazione in Testa (First).	Inserimenti/Estrazioni $\Theta(1)$

Capitolo 11

Algoritmi su Liste Dinamiche e Strutture LIFO

16/03/2026 Implementazione dell'Algoritmo di Giuseppe Flavio

Output 1: Diario Generale di Ripasso (Formato Accademico)

- **Struttura di un problema:** Progettazione e analisi asintotica dell'algoritmo di simulazione per il problema di Giuseppe Flavio, con enfasi sulle operazioni di manipolazione topologica dei puntatori all'interno di una lista circolare.
- **Partiamo da una definizione formale:**
  - **Lista Circolare**[1]
  - **Fase di Inizializzazione**[2]
  - **Fase di Eliminazione (By-pass)**[3]
- **La soluzione:**
  - **Algoritmo di Costruzione:** Viene allocato il primo nodo puntato da T e da X (puntatore di scorrimento). Tramite un ciclo per i da 2 a N, si instanziano i successivi nodi:

$$(*X).succ := Crea_Nodo(i); \quad X := (*X).succ; \quad (*X).succ := T;$$

  - **Algoritmo di Esecuzione del Gioco:** Si utilizza un ciclo `while` la cui condizione di terminazione è la presenza di un solo nodo: $X \neq (*X).succ$.
  - All'interno, un ciclo `for` conta esattamente $M - 1$ passi per posizionarsi sul nodo *precedente* a quello da eliminare: `X := (*X).succ`.
  - L'eliminazione logica (il nodo viene scollegato dall'anello) costa $\Theta(1)$ ed è realizzata dalla seguente istruzione di by-pass:

$$(*X).succ := (*(*X).succ).succ$$

[1] **Lista Circolare:** Configurazione di una lista concatenata in cui il campo puntatore dell'ultimo elemento (*succ*) punta all'indirizzo del primo nodo, formando una topologia ad anello chiuso.

[2] **Fase di Inizializzazione:** Processo iterativo di creazione di N nodi in memoria dinamica, in cui il nodo i-esimo viene collegato al nodo $i + 1$, e l'ultimo viene collegato al primo.

[3] **Fase di Eliminazione (By-pass):** Scorrimento iterativo del puntatore di accesso per $M - 1$ passi, seguito dall'aggiornamento del puntatore del nodo precedente alla vittima per ricollegarlo al nodo successivo alla vittima.

- **Analisi di Complessità:** Il costo totale della fase di eliminazione su lista circolare è limitato superiormente da $\Theta(N \cdot M)$. Qualora si utilizzasse un vettore statico, marcando gli elementi eliminati con un flag logico, l'attraversamento progressivo delle celle marcate (inutili) porterebbe a un degrado quadratico delle prestazioni, valutabile in $\Theta(N^2)$.

Guida di Studio Approfondita e Semplificata (Formato Apprendimento)

Il Concetto Base: L'implementazione algoritmica del problema di Giuseppe Flavio richiede di tradurre in codice un "cerchio" di persone che contano. La lista circolare è perfetta: teniamo traccia di un "bigliettino" (il puntatore X) che passiamo di mano in mano. La regola d'oro in informatica quando si devono eliminare nodi da una lista è: *fermarsi un passo prima.* Se devo eliminare la persona numero 3, mi devo fermare sulla persona numero 2. In questo modo, dico alla persona 2 di prendere la mano della persona 4. La persona 3 è automaticamente estromessa dal cerchio.

Analisi Passo-Passo:

1. *L'Anello:* Creiamo il primo nodo. Poi, per $N-1$ volte, creiamo un nodo nuovo, diciamo a quello attuale di puntare a quello nuovo, e diciamo a quello nuovo di puntare al primissimo nodo (chiudendo il cerchio ad ogni iterazione per sicurezza).
2. *La Conta:* Facciamo un salto in avanti per $M-1$ volte.
3. *L'Eliminazione:* Aggiorniamo la freccia del nodo dove ci siamo fermati, dicendole di saltare il nodo successivo e puntare direttamente a quello dopo ancora.
4. *Fine:* Quando la freccia del nodo punta al nodo stesso, è rimasto un solo superstite.

Il "Perché" è importante: Il confronto $\Theta(N \cdot M)$ della lista circolare contro il disastroso $\Theta(N^2)$ del vettore statico non è un dettaglio accademico: dimostra che un "vettore logico circolare" costringerebbe il processore a percorrere i "fantasmi" delle persone già eliminate, sprecando quantità colossali di cicli di clock verso la fine del gioco. La manipolazione topologica dei puntatori risolve il problema.

Ripasso Visivo/Schematico:

Operazione	Pseudocodice Fondamentale	Significato Fisico
Chiusura Anello	`(*X).succ := T`	L'ultimo nodo chiude il cerchio puntando al primo.
Avanzamento	`X := (*X).succ`	Passaggio di testimone (salto al prossimo superstite).
Bypass (Morte)	`(*X).succ := (*(*X).succ).succ`	Salta il nodo vittima, collegandosi a quello successivo.

16/03/2026 Strutture ad Accesso Ristretto: La Pila (Stack)

Output 1: Diario Generale di Ripasso (Formato Accademico)

- **Struttura di un problema:** Specializzazione delle strutture dati lineari tramite l'imposizione di un vincolo operativo rigido che limita le operazioni di inserimento e rimozione a una singola estremità, garantendo efficienza asintotica ottimale in particolari contesti algoritmici (es. parsing, ricorsione). *(Nota: la controparte Coda/Queue è citata introduttivamente a lezione ma i dettagli operativi e implementativi affrontati si concentrano qui specificamente sulla Pila).*

- **Partiamo da una definizione formale:**
  - **Pila (Stack)**[4]
  - **Pop**[5]
  - **Push**[6]
  - **Top**[7]

- **La soluzione:**
  - **Implementazione mediante Lista Semplice:** Invece di usare liste doppie o strutture complesse, si modella la Pila facendo coincidere la Cima (Top) con il primo nodo della lista concatenata (il nodo puntato dalla testa S).
  - L'operazione `Push(S, a)` alloca un nuovo nodo X, assegna a al campo dati, collega X all'attuale lista ($(*X).succ := S$) e aggiorna il puntatore di testa ($S := X$).
  - L'operazione `Pop(S)` include la gestione delle anomalie. Se la Pila è vuota (condizione `S = nil`), restituisce un valore indeterminato (es. -1 o $\perp$). Altrimenti, la Cima viene aggiornata al nodo successivo:
    $$S := (*S).succ$$
    Il nodo precedente viene isolato e liberato (gestito tramite Garbage Collection o reinserito nella freelist).
  - **Analisi di Complessità:** Data l'assenza totale di iterazioni di attraversamento, ogni singola operazione primitiva (`Push`, `Pop`, `Top`, `IsEmpty`) viene eseguita in un numero strettamente limitato di passi base, ottenendo:
    $$T = \Theta(1)$$

Guida di Studio Approfondita e Semplificata (Formato Apprendimento)

Il Concetto Base: La Pila è l'emblema dell'ordine gerarchico inverso: pensate al caricatore di una pistola o a una pila di piatti in mensa. L'ultimo piatto che il cuoco appoggia in cima alla colonna sarà matematicamente il primo piatto che il cliente prenderà (LIFO: Last In, First Out). In una lista concatenata normale siete liberi di cercare o rimuovere elementi in mezzo; nella Pila questa libertà vi viene sottratta forzatamente per garantirvi in cambio una velocità estrema e una rigida logica di servizio.

Analisi Passo-Passo: Implementare una Pila usando una lista con i puntatori è banale ma elegantissimo.

[4]**Pila (Stack):** Contenitore di informazione omogenea normato dalla disciplina di servizio LIFO (Last In, First Out). Gli elementi entrano ed escono esclusivamente da un'estremità denominata *Top* (Cima).

[5]**Pop:** Operazione di rimozione attiva. Modifica la struttura dati estraendo l'elemento in cima. Funge sovente da operazione combinata: rimuove il nodo e ne restituisce il contenuto associato (*Right Value*).

[6]**Push:** Operazione di inserimento di un nuovo elemento nella posizione di cima.

[7]**Top:** Operazione di ispezione non distruttiva. Restituisce il valore del nodo in cima senza alterare la topologia della struttura.

1. *La Cima:* Decidiamo che la "testa" della lista funge da Cima.
2. *Push (Inserimento):* Quando arriva un nuovo elemento, lo mettiamo fisicamente davanti al primo nodo della lista, aggiornando il puntatore di inizio. Niente scorrimenti necessari.
3. *Pop (Cancellazione):* Leggiamo il valore del primissimo nodo, spostiamo il puntatore di inizio sul secondo nodo e distruggiamo il primo. Anche qui, non serve guardare il resto della lista.
4. *Errore Logico:* Se cerchiamo di fare `Pop` ma il puntatore di testa è `nil`, la macchina non può operare (pila vuota) e genera un messaggio di errore o un valore speciale.

Il "Perché" è importante: Il computer stesso utilizza una Pila (chiamata "Call Stack" o Pila di Esecuzione) per gestire le funzioni, le procedure ricorsive e le variabili locali dei nostri programmi. Ad ogni chiamata, crea un Record di Attivazione e lo impila (`Push`). Quando la funzione termina col comando `return`, estrae il blocco (`Pop`) e il controllo torna esattamente a chi stava sotto di lui. Efficienza fissa in $\Theta(1)$ significa che le operazioni di basso livello del processore non subiranno mai ritardi.

Ripasso Visivo/Schematico:

Operazione Pila	**Analogo su Lista (Testa)**	**Complessità**
`IsEmpty(S)`	Controllo `S = nil`	Costante: $\Theta(1)$
`Push(S, a)`	Inserimento in Testa	Costante: $\Theta(1)$
`Pop(S)`	Rimozione in Testa	Costante: $\Theta(1)$
`Top(S)`	Proiezione in Testa	Costante: $\Theta(1)$

Capitolo 12

Implementazione delle Strutture Dati e Teoria dei Grafi

17/03/2026 Liste Dinamiche: Inserimento e Cancellazione Posizionale

Output 1: Diario Generale di Ripasso (Formato Accademico)

- **Struttura di un problema:** Definizione e implementazione algoritmica delle operazioni di alterazione topologica (inserimento e rimozione) su una lista concatenata dinamica in una generica posizione i.
- **Partiamo da una definizione formale:**
  - **Inserimento in posizione i**[1]
  - **Cancellazione in posizione i**[2]
  - **Vincolo di Indice**[3]
- **La soluzione:**
  - **Gestione del caso base (Testa, $i = 1$):** Sia in caso di inserimento che di cancellazione, operare sulla testa della lista è un'operazione a costo costante $\Theta(1)$. L'inserimento aggiorna il puntatore base della lista al nuovo nodo creato, il quale punterà alla vecchia testa.
  - **Gestione del caso generale (Centro/Coda, $i > 1$):** L'esecutore non può accedere direttamente al nodo. Deve utilizzare un puntatore ausiliario, denominato `pred`, e scorrere la lista per $i - 1$ passi.
  - *Algoritmo di Inserimento Centrale:* Viene istanziato il nuovo nodo in memoria. Il suo puntatore di continuazione (`Next`) eredita la connessione del predecessore. Successi-

[1] **Inserimento in posizione i:** Aggiunta di un nuovo nodo all'interno di una lista esistente in modo che assuma logicamente l'indice i. Richiede il ricollegamento dei puntatori del nodo $i - 1$ (predecessore) e del nodo i originario (successore).

[2] **Cancellazione in posizione i:** Isolamento e rimozione del nodo situato all'indice i. Il predecessore (in posizione $i - 1$) viene ricollegato direttamente al successore (in posizione $i + 1$).

[3] **Vincolo di Indice:** Le operazioni sono lecite solo se $1 \leq i \leq n + 1$ per l'inserimento, e $1 \leq i \leq n$ per la cancellazione, dove n è la cardinalità corrente della lista.

vamente, il predecessore aggiorna la sua connessione per puntare al nuovo nodo.

$$\begin{cases} \texttt{nuovo.Next} \leftarrow \texttt{pred.Next} \\ \texttt{pred.Next} \leftarrow \texttt{nuovo} \end{cases}$$

– *Algoritmo di Cancellazione Centrale:* Individuato il predecessore in posizione $i-1$, si scavalca il nodo vittima assegnando al puntatore del predecessore l'indirizzo del nodo successivo alla vittima. Il nodo orfano viene affidato al meccanismo di Garbage Collection.

$$\texttt{pred.Next} \leftarrow \texttt{pred.Next.Next}$$

Guida di Studio Approfondita e Semplificata (Formato Apprendimento)

Il Concetto Base: Lavorare con le liste concatenate al centro o in coda è come cercare una specifica persona in una staffetta. Dato che i computer non sanno a priori dove si trovi la persona in posizione 5, devono partire dal primo e seguire la catena delle mani fino ad arrivare alla persona 4 (il predecessore). È sempre fondamentale fermarsi "un passo prima" del punto in cui si vuole operare, per poter sganciare e riagganciare correttamente i collegamenti.

Analisi Passo-Passo: Cosa succede fisicamente quando si esegue un inserimento al centro?

1. Creiamo il nodo "Nuovo" nella memoria dinamica (ha un suo spazio, ma è staccato dal mondo).
2. Mandiamo una sonda (`pred`) a camminare per la lista fino al nodo $i-1$.
3. Diciamo al "Nuovo" nodo di prendere la mano di colui che originariamente era in posizione i (salvando quindi il resto della lista).
4. Diciamo alla sonda in posizione $i-1$ di lasciare la mano vecchia e prendere quella del "Nuovo" nodo. L'ordine dei passaggi 3 e 4 è cruciale: se invertito, perderemmo per sempre l'indirizzo del resto della lista!

Il "Perché" è importante: Il vero limite di questa operazione è lo scorrimento iniziale. A differenza degli array, dove accedere alla cella i richiede tempo $\Theta(1)$, qui il computer deve attraversare la lista per $i-1$ iterazioni, spingendo la complessità computazionale asintotica verso il caso pessimo lineare, ovvero $\Theta(n)$.

Ripasso Visivo/Schematico:

Azione	**Posizione i**	**Lavoro Meccanico sui Puntatori**
Inserimento	$i=1$	`nuovo.Next = Testa;` `Testa = nuovo`
Inserimento	$i>1$	`nuovo.Next = pred.Next;` `pred.Next = nuovo`
Cancellazione	$i=1$	`Testa = Testa.Next`
Cancellazione	$i>1$	`pred.Next = pred.Next.Next`

17/03/2026 Implementazione Dinamica di Pile e Code

Output 1: Diario Generale di Ripasso (Formato Accademico)

- **Struttura di un problema:** Implementare collezioni ad accesso ristretto (Pila LIFO e Coda FIFO) su supporto interamente dinamico tramite riferimenti a memoria, eliminando il vincolo della dimensione prefissata intrinseco all'uso dei vettori e garantendo complessità costante per ogni manipolazione.
- **Partiamo da una definizione formale:**
  - **Pila Dinamica (Stack)**[4]
  - **Coda Dinamica (Queue)**[5]
- **La soluzione:**
  - **Stack (LIFO):** L'inserimento (`push`) agisce esclusivamente in testa al fine di mantenere il costo temporale in $\Theta(1)$:

    $$\texttt{oldfirst} \leftarrow \texttt{first}; \quad \texttt{first} \leftarrow \text{Nuovo Nodo}; \quad \texttt{first.Next} \leftarrow \texttt{oldfirst}$$

    Analogamente, l'estrazione (`pop`) riposiziona il *Top* al successore, restituendo il dato del nodo sganciato.
  - **Queue (FIFO):** L'inserimento (`enqueue`) deve avvenire in coda. Avendo salvato esplicitamente il puntatore `last`, l'operazione evita lo scorrimento $\Theta(n)$ della lista, limitandosi a:

    $$\texttt{oldlast.Next} \leftarrow \texttt{last} \quad \text{(dopo aver istanziato il nuovo last)}$$

    Particolare attenzione è rivolta all'inserimento del primo elemento assoluto (`isEmpty == true`): in tale singolarità, sia `first` che `last` devono essere inizializzati a puntare al medesimo (e unico) nodo istanziato.

Guida di Studio Approfondita e Semplificata (Formato Apprendimento)

Il Concetto Base: Per simulare le restrizioni della Pila e della Coda, prendiamo una normale lista e ci mettiamo delle limitazioni.

- Nella **Pila**, ci serve usare un solo dito: il puntatore alla cima (`first`). Qualsiasi operazione avviene sul primo elemento, rendendo tutto istantaneo.
- Nella **Coda**, se avessimo solo il puntatore alla testa, per inserire un nuovo cliente dovremmo scorrere tutta la fila dal primo all'ultimo. Per evitare questa lentezza, la Coda è programmata per tenere in memoria sempre due puntatori: uno sul primo (`first` per servire) e uno fisso sull'ultimo (`last` per inserire velocemente in fondo).

Analisi Passo-Passo: Il problema più subdolo da gestire nello sviluppo di una Coda è l'inserimento in una "Coda Vuota". Normalmente, l'inserimento (`enqueue`) prende l'ultimo della fila e gli aggancia dietro il nuovo arrivato. Ma se la fila è vuota, l'ultimo non esiste! Il programma deve accorgersi che la coda è vuota, creare il nodo e dire: "Poiché sei l'unico cliente presente, tu sei logicamente sia il primo da servire (`first`), sia l'ultimo della fila (`last`)". Dal secondo cliente in poi, il codice torna a comportarsi in modo standard (aggiornando solo il `last`).

[4] **Pila Dinamica (Stack):** Implementazione basata su una lista singolarmente concatenata dotata di un unico puntatore d'accesso denominato `first` (che identifica inequivocabilmente il *Top*).

[5] **Coda Dinamica (Queue):** Implementazione basata su lista concatenata dotata di due puntatori persistenti e distinti: `first` (testa logica e punto di prelievo) e `last` (coda logica e punto di inserimento).

Il "Perché" è importante: Sfruttare le liste concatenate dinamiche per Pile e Code risolve il grave problema dell'overflow (la coda piena) tipico dell'implementazione via Array. Con l'utilizzo combinato del puntatore in coda (`last`), anche l'accesso sequenziale scompare, regalando una complessità eccezionale pari a $\Theta(1)$ per ogni singola transazione, per sempre, fino all'esaurimento della RAM fisica del computer.

Ripasso Visivo/Schematico:

Struttura	**Puntatori Mantenuti**	**Inserimento**	**Rimozione**
Pila (Stack)	Solo `first`	In testa su `first`	Dalla testa su `first`
Coda (Queue)	`first` e `last`	In fondo usando `last`	Dalla testa usando `first`

17/03/2026 Introduzione alla Teoria dei Grafi

Output 1: Diario Generale di Ripasso (Formato Accademico)

- **Struttura di un problema:** Definire un modello matematico astratto capace di rappresentare la topologia di relazioni binarie generiche su un insieme di entità (es. reti di comunicazione, mappe, interazioni sociali).
- **Partiamo da una definizione formale:**
  - **Grafo Non Orientato**[6]
  - **Grafo Orientato**[7]
- **La soluzione:**
  - L'utilizzo del prodotto cartesiano definisce i confini delle relazioni: in un grafo non orientato, il lato $\{A, B\}$ è indistinguibile e logicamente identico a $\{B, A\}$.
  - Nel grafo orientato, al contrario, gli archi (A, B) e (B, A) sono entità matematiche distinte in V^2. Un esempio fisico illustrato a lezione riguarda l'astrazione della mappa di una rete metropolitana: se i treni percorrono i binari da A a B e parallelamente da B ad A in binari separati, il sistema soggiacente è propriamente un grafo orientato con due archi distinti, che può essere astratto in un grafo non orientato macroscopico quando la distinzione fisica dei binari viene ignorata a favore del concetto unificato di "collegamento".

Guida di Studio Approfondita e Semplificata (Formato Apprendimento)

Il Concetto Base: Cosa sono i grafi? Semplicemente dei punti (i **vertici** o nodi) uniti da linee (gli **archi** o lati). Se i grafi sono *non orientati*, le linee sono come delle normali strade a doppio senso di marcia: il collegamento tra la città A e la città B è uno solo, usato per andare e tornare indistintamente. Se i grafi sono *orientati*, le linee sono delle frecce, come strade a senso unico. Il fatto che io possa guidare da A a B (arco (A, B)) non mi garantisce affatto che io possa tornare

[6] **Grafo Non Orientato:** Una coppia $G = \langle V, E \rangle$ dove V è l'insieme finito dei vertici (o nodi) ed $E \subseteq V^{(2)}$ è l'insieme dei lati. Un lato è una coppia non ordinata di vertici (la relazione è simmetrica).

[7] **Grafo Orientato:** Una coppia $G = \langle V, E \rangle$ dove $E \subseteq V^2$ è un insieme di archi. Un arco è una coppia rigorosamente ordinata (u, v), definendo una relazione con una specifica direzione (da sorgente a destinazione).

indietro; se voglio farlo, l'amministrazione deve costruire una seconda strada a senso unico da B ad A (arco (B, A)).

Analisi Passo-Passo: Durante la lezione è stato fatto il caso della mappa di una metropolitana. A prima vista sembra un grafo non orientato (una linea che unisce le stazioni). Tuttavia, se ci addentriamo nell'infrastruttura, un treno usa un binario per andare e un binario diverso e fisicamente separato per tornare. A livello ingegneristico si tratta di due percorsi *orientati* opposti (archi). A livello di mappa turistica si disegna una singola linea, astraendo verso il concetto simmetrico di grafo *non orientato* (lato).

Il "Perché" è importante: Scegliere tra grafo orientato o non orientato cambia radicalmente la matematica e l'algoritmica applicata in seguito. Algoritmi per il calcolo di percorsi minimi o alberi di copertura possono funzionare bene nel caso non orientato ma fallire catastroficamente se le strade sono a senso unico.

Ripasso Visivo/Schematico:

Tipologia	**Nome Collegamento**	**Struttura Matematica**	**Simmetria**
Non Orientato	Lato (Edge)	Insieme $\{u, v\}$	Sì ($\{A, B\} \equiv \{B, A\}$)
Orientato	Arco (Arc)	Coppia ordinata (u, v)	No ($(A, B) \neq (B, A)$)

Capitolo 13

Alberi: Topologia, Proprietà Matematiche e Combinatoria

19/03/2026 Tassonomia e Struttura: Alberi Ordinati e Binari

Output 1: Diario Generale di Ripasso (Formato Accademico)

- **Struttura di un problema:** Sviluppo di una rigorosa tassonomia per le strutture dati non lineari ad albero, imponendo vincoli relazionali e posizionali progressivi per facilitarne l'astrazione, l'implementazione in memoria e la successiva visita algoritmica.
- **Partiamo da una definizione formale:**
  - **Albero con Radice Ordinato (ARO)**[1]
  - **Albero Binario**[2]
  - **Albero Degenere**[3]
- **La soluzione:**
  - L'imposizione della differenziazione posizionale nell'albero binario altera il concetto di uguaglianza strutturale. Un albero formato da una radice avente unicamente un figlio sinistro è da considerarsi matematicamente e logicamente distinto da un albero avente la stessa radice e unicamente un figlio destro.
  - Questa rigidità risolve il problema della mappatura in memoria centrale. L'albero binario si traduce naturalmente in un record (o nodo) contenente il dato e due riferimenti di memoria (puntatori) fissi ed espliciti: `Sx` (sinistro) e `Dx` (destro), che possono eventualmente assumere valore nullo (`nil`).

[1] **Albero con Radice Ordinato (ARO):** Un albero radicato all'interno del quale è definito un ordine totale su ogni classe di equivalenza generata dalla relazione "essere fratelli". Questa struttura impone una precisa distinzione posizionale (primo figlio, secondo figlio, n-esimo figlio) tra i nodi discendenti di un medesimo padre.

[2] **Albero Binario:** Un albero con radice ordinato in cui il grado uscente massimo di ogni nodo è limitato superiormente a 2. I figli sono rigorosamente e semanticamente distinti in "figlio sinistro" e "figlio destro".

[3] **Albero Degenere:** Caso topologico limite di un albero binario in cui la ramificazione è del tutto assente. È composto da n nodi e possiede un'altezza pari a $n - 1$, risultando isomorfo a una lista concatenata.

Guida di Studio Approfondita e Semplificata (Formato Apprendimento)

Il Concetto Base: Pensate a un albero genealogico classico. Se Marco e Luca sono fratelli, l'albero mostra semplicemente che discendono dagli stessi genitori. In informatica, questo è un "albero con radice" normale. Ma i computer hanno bisogno di ordini precisi: l'Albero con Radice Ordinato mette in fila indiana Marco e Luca, stabilendo chi viene prima e chi dopo. Se stringiamo ulteriormente le regole e imponiamo la legge del "massimo due figli per nodo", obbligando inoltre a chiamare il primo figlio "Sinistro" e il secondo "Destro", abbiamo appena creato l'**Albero Binario**.

Analisi Passo-Passo: Perché l'albero binario considera distinti un figlio sinistro e un figlio destro anche se sono "figli unici"? Immaginate una macchina che deve viaggiare esplorando l'albero. Se diciamo alla macchina "Vai a sinistra", e il figlio è salvato a destra, la macchina cadrà nel vuoto e troverà un errore (riferimento nullo). Mantenere distinti i due rami garantisce che le operazioni di esplorazione siano deterministiche e prive di ambiguità. Se questa biforcazione fallisce per ogni singolo nodo, e l'albero cresce solo in una singola direzione, si parla di **Albero Degenere**, che non è altro che una lista concatenata sotto mentite spoglie.

Il "Perché" è importante: Limitare il grado a 2 (Albero Binario) standardizza la creazione della struttura in memoria. Ogni nodo saprà sempre di aver bisogno di esattamente 2 "frecce" (puntatori) per i suoi successori. Questa prevedibilità stabilizza la complessità spaziale e rende i linguaggi di programmazione capaci di esplorare gerarchie enormi in modo ricorsivo.

Ripasso Visivo/Schematico:

Tipologia	Ordine tra Fratelli?	Caratteristica Chiave
Albero Radicato	No	Gerarchia pura (Padre-Figlio).
Albero Ordinato	Sì	Esiste un 1°, 2°... k-esimo figlio.
Albero Binario	Sì (Estremo)	Max 2 figli, rigida etichettatura (Sx, Dx) anche per figli unici.

19/03/2026 Proprietà Matematiche, Serie Geometrica e Combinatoria

Output 1: Diario Generale di Ripasso (Formato Accademico)

- **Struttura di un problema:** Analisi quantitativa e valutazione combinatoria dello spazio degli stati topologici degli alberi. L'obiettivo è determinare l'incidenza dei casi limite (strutture degeneri vs. strutture sature) per derivare i limiti prestazionali teorici degli algoritmi di visita.
- **Partiamo da una definizione formale:**
  - **Serie Geometrica Troncata**[4]
  - **Albero k-ario Completo**[5]
  - **Combinatoria del Caso Degenere**[6]

[4] **Serie Geometrica Troncata:** Sommatoria delle potenze successive di una base x, definita matematicamente come $f_n(x) = \sum_{i=0}^{n} x^i$. Per $x > 1$, ammette la soluzione chiusa: $\frac{x^{n+1}-1}{x-1}$.

[5] **Albero k-ario Completo:** Un albero in cui ogni nodo interno ha esattamente k figli e tutte le foglie si trovano al medesimo livello topologico h. Il numero massimo di nodi è modellato proprio dalla serie geometrica troncata $f_h(k)$.

[6] **Combinatoria del Caso Degenere:** La determinazione della cardinalità dell'insieme di tutti i possibili alberi binari degeneri componibili a partire da un numero di nodi n prefissato.

- **La soluzione:**
  - **Calcolo dei nodi nel caso ottimo (Albero Saturo):** Applicando la serie geometrica a un albero ternario ($k = 3$) di altezza h, la capienza massima (il numero di nodi n) viene valutata mediante la formula esplicita:

    $$f_h(3) = \frac{3^{h+1} - 1}{2}$$

    L'altezza h esibisce pertanto una crescita logaritmica $h \sim \log_k n$, garantendo tempi di accesso asintoticamente ideali $\Theta(\log n)$.
  - **Calcolo della cardinalità del caso pessimo (Alberi Degeneri):** La dimostrazione procede istituendo una corrispondenza biunivoca (isomorfismo strutturale) tra la topologia dell'albero degenere e l'insieme delle stringhe binarie.
    1. Fissata la radice dell'albero degenere, restano $n - 1$ nodi da collocare.
    2. Per ciascuno di questi $n-1$ livelli topologici discendenti, esiste una singola e unica decisione binaria da compiere: innestare il nodo a sinistra (bit 0) oppure a destra (bit 1).
    3. Essendoci 2 scelte indipendenti per $n - 1$ passaggi consecutivi, il calcolo combinatorio delle disposizioni con ripetizione restituisce esattamente:

    $$C_{degenere}(n) = \underbrace{2 \times 2 \times \cdots \times 2}_{n-1 \text{ volte}} = 2^{n-1}$$

    Il numero di aberrazioni strutturali cresce quindi asintoticamente in misura esponenziale $O(2^n)$.

Guida di Studio Approfondita e Semplificata (Formato Apprendimento)

Il Concetto Base: Quanto può essere "pieno" o "vuoto" un albero informatico? - Se è "pieno" (tutti i nodi hanno il massimo dei figli possibili), il numero di elementi esplode a ogni piano che scendiamo. Se ogni nodo fa 3 figli, partiamo da 1 radice, poi 3 figli, poi 9 nipoti, poi 27 pronipoti... Sommare tutti questi numeri per sapere quanto è grande l'albero è noioso. La *Serie Geometrica* ci dà una formula matematica chiusa per saltare il conteggio manuale e ottenere il totale in una frazione di secondo. - Se l'albero è "vuoto", ossia ogni nodo fa un solo figlio, otteniamo l'Albero Degenere (una fila indiana di nodi). Ma in quante forme diverse possiamo creare questa fila indiana in un albero binario?

Analisi Passo-Passo: Capiamo perché gli alberi degeneri di 1000 nodi sono un numero mastodontico:

1. Posiziona la radice (il primo nodo).
2. Per attaccare il 2° nodo, hai 2 scelte: Sinistra o Destra.
3. Dal 2° nodo, per attaccare il 3°, hai altre 2 scelte: Sinistra o Destra.

Se ripeti questo ragionamento per tutti i restanti 999 nodi, le combinazioni si moltiplicano fra loro: $2 \times 2 \times 2 \ldots$ calcolato 999 volte. Il risultato è 2^{999}, un numero gigantesco, di gran lunga superiore al numero di atomi nell'universo visibile!

Il "Perché" è importante: Sapere che ci sono così tanti alberi degeneri (il caso peggiore in assoluto per le performance, perché obbliga il computer a scorrere tutto l'albero in lentezza lineare $\Theta(n)$) potrebbe spaventare. Tuttavia, se confrontato al numero totale e astronomico di

tutti i possibili alberi sani che possiamo creare, l'albero degenere rimane un'eccezione statistica gestibile dalle euristiche avanzate di bilanciamento (che si studiano nelle strutture come i B-Tree o gli alberi Red-Black).

Ripasso Visivo/Schematico:

Forma	**Proprietà della Ramificazione**	**Altezza h in funzione di n**
Albero Pieno (Completo)	Espansione massimizzata basata sulla Serie Geometrica $\sum k^i$.	Ottima: $h \sim \log_k n$
Albero Degenere	Singola scelta (0/1) iterata $n-1$ volte. Numero di configurazioni: 2^{n-1}.	Pessima: $h = n - 1$

Capitolo 14

Proprietà degli Alberi e Attraversamento dei Grafi

23/03/2026 Alberi Localmente Completi e Numeri di Catalan

Output 1: Diario Generale di Ripasso (Formato Accademico)

- **Struttura di un problema:** Determinazione dello spazio degli stati topologici per una specifica classe di alberi binari e dimostrazione del legame matematico invariante tra le lunghezze dei cammini interni ed esterni.
- **Partiamo da una definizione formale:**
  - **Albero Binario Localmente Completo**[1]
  - **Lunghezza del Cammino Interno (LC_i)**[2]
  - **Lunghezza del Cammino Esterno (LC_e)**[3]
  - **Numeri di Catalan**[4]
- **La soluzione:**
  - **Costruzione dell'equazione di ricorrenza per i Numeri di Catalan (C_n):** Fissata la radice (che è il primo nodo interno), i restanti $n-1$ nodi interni devono essere ripartiti tra il sottoalbero sinistro e il sottoalbero destro. Se il sottoalbero sinistro riceve $k-1$ nodi interni, il destro ne riceverà $n-k$. Moltiplicando le combinazioni possibili si ottiene la ricorrenza:

$$c_n = \sum_{k=1}^{n} c_{k-1} c_{n-k} \quad \text{con } c_0 = 1$$

[1] **Albero Binario Localmente Completo:** Un albero binario in cui ogni nodo interno possiede rigorosamente e in ogni caso esattamente due figli (grado uscente pari a 2).

[2] **Lunghezza del Cammino Interno (LC_i):** La somma delle profondità (o livelli) di tutti i nodi interni dell'albero.

[3] **Lunghezza del Cammino Esterno (LC_e):** La somma delle profondità (o livelli) di tutti i nodi foglia dell'albero.

[4] **Numeri di Catalan:** Successione matematica che descrive il numero di possibili configurazioni strutturali di un albero binario localmente completo avente n nodi interni.

La cui soluzione chiusa, basata sul coefficiente binomiale, è:

$$c_n = \frac{1}{n+1}\binom{2n}{n}$$

- **Relazione tra LC_e e LC_i:** In un albero binario localmente completo con n nodi interni vale sempre la relazione:

$$LC_e = LC_i + 2n$$

  Dimostrazione costruttiva: Partendo da un albero con sola radice ($LC_e = 0, LC_i = 0$), ad ogni passo si seleziona una foglia a profondità k e la si trasforma in nodo interno aggiungendo due nuovi figli (foglie) a profondità $k+1$. Il cammino interno incrementa di k. Il cammino esterno perde la foglia a livello k e guadagna due foglie a livello $k+1$, subendo una variazione di $-k + 2(k+1) = k+2$. Iterando per n passi si dimostra l'invariante.
- *Nota sulla frammentarietà logica (Vettore dei Padri):* Le registrazioni analizzano la rappresentazione in memoria tramite array dei padri. Si segnala che sebbene il vettore dei padri definisca perfettamente le adiacenze, fallisce nel preservare la semantica degli *alberi con radice ordinati*, poiché perde l'informazione cruciale sulla distinzione topologica tra figlio sinistro e figlio destro.

Guida di Studio Approfondita e Semplificata (Formato Apprendimento)

Il Concetto Base: Immaginate di dover costruire una rete idrica o un organigramma aziendale con una regola ferrea: ogni capo reparto deve avere esattamente due sottoposti diretti. Mai uno, mai tre. Quante forme diverse può assumere questa azienda se sappiamo che ci sono n capi reparto? La matematica risponde a questa domanda con i "Numeri di Catalan". Inoltre, in questi alberi perfetti, esiste una bilancia magica: la distanza totale di tutte le "foglie" (i dipendenti base) dalla cima dell'azienda è sempre uguale alla distanza totale di tutti i "capi" dalla cima, più il doppio del numero dei capi.

Analisi Passo-Passo: Come si dimostra la regola dei cammini ($LC_e = LC_i + 2n$)?

1. Immaginiamo l'albero come un seme (una sola foglia/radice). Cammini = 0.
2. Facciamo germogliare una foglia: essa diventa un ramo (nodo interno) e genera due nuove foglioline.
3. Se la foglia era al livello 5, il cammino dei nodi interni cresce di 5.
4. Il cammino delle foglie perde quel 5 (perché non è più foglia), ma acquista due nuove foglie al livello 6 (quindi +12). Il guadagno netto è $12 - 5 = 7$.
5. Notate che 7 è esattamente $5 + 2$. Quindi, ad ogni singolo passo di crescita, il cammino esterno cresce sempre di "2 in più" rispetto al cammino interno. Ripetendo per n passi, il divario diventa esattamente $2n$.

Il "Perché" è importante: Il calcolo combinatorio dei Numeri di Catalan ci avvisa di quanto rapidamente esploda il numero di configurazioni strutturali di un albero. Questo è vitale per l'analisi dei compilatori e degli algoritmi di decisione, dove ogni albero localmente completo rappresenta ad esempio una sequenza di operazioni matematiche valide su n variabili.

Ripasso Visivo/Schematico:

Concetto	Formula Matematica	Significato Fisico
Spazio delle Topologie	$C_n = \frac{1}{n+1}\binom{2n}{n}$	Numero di alberi possibili dati n nodi interni.
Invariante dei Cammini	$LC_e = LC_i + 2n$	Bilancio esatto tra profondità di foglie e nodi interni.
Vettore dei Padri	`padre[figlio] = ID`	Semplice ma incapace di codificare l'ordine (Sx/Dx).

23/03/2026 Strategie di Visita dei Grafi: Ampiezza e Profondità

Output 1: Diario Generale di Ripasso (Formato Accademico)

- **Struttura di un problema:** Attraversamento sistematico e deterministico di tutti i nodi di un grafo connesso (e non orientato) a partire da un vertice sorgente s, evitando di intrappolare l'esecutore in cicli infiniti.
- **Partiamo da una definizione formale:**
  - **Vettore di Stato (Nuovo/Vecchio)**[5]
  - **Visita in Ampiezza (BFS)**[6]
  - **Visita in Profondità (DFS)**[7]
- **La soluzione:**
  - **Algoritmo BFS (Ampiezza):** Si inserisce la sorgente nella coda e si imposta il suo stato a "Vecchio". Finché la coda non è vuota: si estrae la testa (Front), lo si definisce come nodo corrente, e si ispeziona la sua lista di adiacenza. Ogni vicino "Nuovo" viene inserito in coda e immediatamente marcato come "Vecchio". L'estrazione dalla coda definisce l'ordine di visita.
  - **Algoritmo DFS (Profondità):** L'esecuzione si muove dinamicamente salvando il contesto nella Pila. Il nodo sorgente viene posto nella Pila. Si esamina il primo vicino "Nuovo" nella lista di adiacenza del nodo in cima allo stack, lo si marca come "Vecchio" e lo si inserisce (Push) nello stack.
  - *Condizione di Ritorno (Backtracking):* Se il nodo corrente (Top della Pila) esaurisce la sua lista di adiacenza senza trovare nodi "Nuovi" (i vicini sono già stati tutti visitati), la procedura esegue una Pop della Pila. Questa estrazione riporta la macchina al nodo precedentemente esplorato, riprendendo a scorrere la sua lista di adiacenza dall'esatto punto in cui era stata sospesa.

[5] **Vettore di Stato (Nuovo/Vecchio):** Vettore logico parallelo ai vertici del grafo in cui ogni nodo viene marcato inizialmente a 1 (Nuovo) e successivamente posto a 0 (Vecchio/Visitato) per garantire che venga processato una sola volta.

[6] **Visita in Ampiezza (BFS):** Esplorazione a raggiera. Visita tutti i nodi a distanza 1 dalla sorgente, poi tutti quelli a distanza 2, ecc. Richiede obbligatoriamente l'uso di una struttura dati Coda (FIFO) per gestire le frontiere di esplorazione.

[7] **Visita in Profondità (DFS):** Esplorazione in allungo. Cerca di prolungare il cammino corrente visitando un nodo adiacente non ancora esplorato. In assenza di adiacenze valide, si innesca il *backtracking* (ritorno sui propri passi). Implementabile ricorsivamente o tramite l'uso di una Pila (LIFO).

- **Complessità:** L'operazione richiede un tempo strettamente dipendente dalla rappresentazione in memoria. Per le liste di adiacenza è $\Theta(N+M)$ (nodi + lati). Lo spazio richiesto è governato dai vettori di stato e dalla dimensione massima assunta dalla Coda/Pila, quantificabile in $\Theta(N)$.

Guida di Studio Approfondita e Semplificata (Formato Apprendimento)

Il Concetto Base: Immaginate di essere all'ingresso di un grande labirinto circolare (il Grafo) e di dover visitare tutte le stanze. Dato che il labirinto ha stanze collegate in tondo (cicli), vi serve un secchio di vernice per fare una X sulle porte delle stanze in cui siete già stati (il vettore *Nuovo/Vecchio*), altrimenti girereste in tondo per l'eternità. Potete procedere in due modi:

- *In Ampiezza (BFS):* Mandate dei cloni a guardare tutte le stanze a 1 passo di distanza. Poi a 2 passi. È come l'onda generata da un sasso in uno stagno che si allarga.
- *In Profondità (DFS):* Andate dritti per un corridoio finché non trovate un muro o stanze già pitturate. A quel punto fate marcia indietro (Backtracking) fino all'ultimo bivio che avevate ignorato e prendete un'altra strada.

Analisi Passo-Passo: A lezione è stato tracciato visivamente il funzionamento dei due algoritmi:

1. **Ampiezza:** Usa una **Coda (la fila della posta)**. Inseriamo il nodo A. Togliamo A e inseriamo tutti i suoi vicini (B, C, D). Ora serviamo B: lo togliamo e mettiamo in coda i suoi vicini nuovi. L'effetto è un'espansione a strati omogenei.
2. **Profondità:** Usa uno **Stack (Pila di piatti)**. Mettiamo A. A vede B. Mettiamo B *sopra* A. B vede C. Mettiamo C *sopra* B. Il computer si concentra sempre e solo sul "piatto in cima". Cosa succede se C è un vicolo cieco (i suoi vicini sono tutti vecchi)? Togliamo il piatto C (Pop). Il piatto sottostante torna a essere B. Magicamente, l'algoritmo ha appena fatto un passo indietro, tornando nella stanza B per cercare nuove porte.

Il "Perché" è importante: Scegliere tra queste due visite non è questione di gusti, ma di scopo del software. Se programmate un GPS per trovare la strada col minor numero di incroci per un ospedale, l'algoritmo in Ampiezza è perfetto, perché trova i percorsi a "distanza minima". Se programmate un sistema per esplorare l'intero albero delle mosse degli scacchi (Deep Blue), si usa la Profondità per viaggiare il più lontano possibile e valutare la singola vittoria/sconfitta prima di tornare indietro.

Ripasso Visivo/Schematico:

Strategia	**Struttura Memoria**	**Meccanica di Esplorazione**	**Utilità Principale**
Ampiezza (BFS)	Coda (FIFO)	Frontiere concentriche (livelli).	Calcolo delle distanze minime o cammini minimi (non pesati).
Profondità (DFS)	Pila (LIFO)	Esplora il più a fondo possibile, poi esegue Backtracking.	Esplorazione esaustiva di labirinti, analisi di percorsi terminali.

Capitolo 15

Esplorazione di Grafi e Alberi: Strategie e Complessità

24/03/2026 Dinamica della Visita in Profondità (DFS) su Grafi

Output 1: Diario Generale di Ripasso (Formato Accademico)

- **Struttura di un problema:** Gestione dello scorrimento esaustivo dei nodi di un grafo connesso tramite un approccio esplorativo non a raggiera, bensì basato sull'allungamento massimo del cammino corrente, sfruttando una memoria a disciplina LIFO per il recupero dei bivi inesplorati.
- **Partiamo da una definizione formale:**
  - **Stato del Nodo (Nuovo/Vecchio)**[1]
  - **Pila di Esplorazione (Stack)**[2]
  - **Backtracking**[3]
- **La soluzione:**
  - *(Nota: La traccia di esecuzione sui nodi A, B, C, D... è presentata nell'audio in modo altamente discorsivo e analitico; la sintesi formale del comportamento è la seguente).*
  - Si inserisce la radice/sorgente s nello Stack e la si marca come Vecchia.
  - Iterativamente si ispeziona il nodo in cima (Top) allo Stack. Si analizza la sua lista di adiacenza alla ricerca del *primo* nodo "Nuovo" w.
  - Se w esiste: viene marcato come Vecchio e inserito nello Stack (Push). L'esecuzione si sposta immediatamente su w, sospendendo l'analisi del nodo precedente.
  - Se nessun w esiste (vicolo cieco): il nodo corrente viene rimosso dallo Stack (Pop), realizzando il backtracking.

[1] **Stato del Nodo (Nuovo/Vecchio):** Proprietà logica associata a ogni vertice del grafo, implementata tramite un vettore parallelo booleano. Inizializzato a "Nuovo" (0), transita a "Vecchio" (1) all'atto dell'inserimento nella struttura dati, prevenendo cicli infiniti.

[2] **Pila di Esplorazione (Stack):** Struttura dati LIFO utilizzata per memorizzare i nodi scoperti ma la cui lista di adiacenza non è stata ancora completamente scansionata.

[3] **Backtracking:** Fase dell'algoritmo innescata quando il nodo correntemente in cima alla Pila non possiede alcun nodo adiacente nello stato "Nuovo". Consiste nell'estrazione (Pop) del nodo per riprendere la scansione della lista di adiacenza del nodo sottostante.

- **Analisi di Complessità:** Ogni nodo entra nello Stack esattamente una volta e la sua lista di adiacenza viene percorsa linearmente. La complessità temporale per grafi implementati con liste di adiacenza è proporzionale a nodi e lati:

$$T(n) = \Theta(N + M)$$

Lo spazio occupato è dominato dal vettore di stato e dall'altezza massima dello Stack. Nel caso peggiore (grafo isomorfo a una lista), lo Stack contiene tutti gli N nodi:

$$S(n) = \Theta(N)$$

Guida di Studio Approfondita e Semplificata (Formato Apprendimento)

Il Concetto Base: La Visita in Profondità (DFS) è come esplorare un labirinto bendati, tenendo una mano sempre sul muro destro. Si avanza dritti in un corridoio ignorando momentaneamente i bivi. Si continua a camminare finché non si sbatte contro un muro chiuso (un nodo che non ha vicini) o finché non si arriva in una stanza dove c'è il segno di un gessetto che avevamo lasciato noi stessi (nodo "Vecchio"). A quel punto, si fa un passo indietro (Backtracking) e si imbocca l'ultimo bivio che avevamo ignorato.

Analisi Passo-Passo:

1. *Avanzamento (Push):* Troviamo un nodo nuovo, lo marchiamo col gessetto, lo mettiamo in cima alla Pila dei "posti in cui siamo passati" e ci tuffiamo al suo interno.

2. *Vicolo Cieco:* Il nodo in cima alla Pila guarda tutti i suoi vicini e scopre che sono tutti marcati col gessetto.

3. *Ritirata (Pop):* Togliamo questo nodo inutile dalla Pila. Ora, magicamente, il nodo che si trova in cima è quello precedente! Il computer riprende a ispezionare i vicini del nodo precedente esattamente dal punto in cui era stato interrotto.

Il "Perché" è importante: Il comportamento della Pila (LIFO) ricalca perfettamente la Memoria di Sistema del computer (il Call Stack). Usare una Pila esplicita per gestire questa visita ci permette di esplorare reti immense (es. i link di Internet o le mosse di una partita a scacchi) senza far "esplodere" la memoria del computer, cosa che succederebbe provando a usare la ricorsione nativa su grafi troppo profondi.

Ripasso Visivo/Schematico:

Condizione	Azione su Pila	Significato Topologico
Trovato vicino "Nuovo"	`Push`	Prolungamento del cammino (scendo più in profondità).
Tutti vicini "Vecchi"	`Pop`	Vicolo cieco. Ritiro strategico al nodo precedente (Backtracking).
Pila vuota	`Termine`	Tutte le componenti connesse alla sorgente sono state visitate.

24/03/2026 Strategie di Visita per Alberi con Radice Ordinati

Output 1: Diario Generale di Ripasso (Formato Accademico)

- **Struttura di un problema:** Sfruttare l'assenza di cicli e la gerarchia stretta degli alberi per definire algoritmi di attraversamento sistematici, categorizzati in base al momento esatto in cui l'esecutore processa (visita) il nodo radice rispetto ai suoi sottoalberi.
- **Partiamo da una definizione formale:**
  - **Visita in Ordine Anticipato (Pre-Order)**[4]
  - **Visita in Ordine Simmetrico (In-Order)**[5]
  - **Visita in Ordine Posticipato (Post-Order)**[6]
  - **Visita per Livelli**[7]
- **La soluzione:**
  - **Implementazione Ricorsiva vs Iterativa:** Sebbene l'approccio ricorsivo sia di scrittura banale, l'eliminazione della ricorsione richiede la manipolazione esplicita di uno Stack. *(Nota dall'audio: l'interpretazione sintattica dell'albero tramite parentesi aperte e chiuse illustrata a lezione funge da base per comprendere la serializzazione, omettendo dettagli implementativi complessi a favore del modello mentale).*
  - **Meccanica Iterativa Pre-Order:** Si estrae il nodo corrente dallo Stack e lo si visita. Poi, si inseriscono nello Stack i suoi figli. Per garantire che il figlio sinistro venga estratto per primo, è obbligatorio inserire *prima il figlio destro e poi il figlio sinistro* (grazie alla politica LIFO).
  - **Complessità Asintotica:** Indipendentemente dall'ordine scelto (Pre, In, Post-Order), ogni nodo è inserito nello Stack, elaborato ed estratto un numero costante di volte. Il tempo di calcolo è puramente lineare sul numero totale di nodi n:

    $$T(n) = \Theta(n)$$

    La complessità spaziale è determinata dall'altezza dell'albero h. Nel caso peggiore (Albero Degenere), lo Stack collassa allocando un frame per ogni nodo:

    $$S(n) = O(n) \quad \text{(Limite stringente al caso peggiore: } \Theta(n)\text{)}$$

Guida di Studio Approfondita e Semplificata (Formato Apprendimento)

Il Concetto Base: Per visitare un albero (cioè passare a salutare ogni singolo nodo) dobbiamo decidere quando bussare alla porta del padre. Immaginate una cartella sul computer:

- *Pre-Order (Anticipato):* Leggete il nome della cartella, poi aprite i file dentro. Utile per copiare un intero albero così com'è.
- *Post-Order (Posticipato):* Guardate quanto pesano tutti i file dentro, e solo alla fine calcolate il peso totale della cartella madre. Non potete valutare il padre senza aver prima valutato i figli.

[4]**Visita in Ordine Anticipato (Pre-Order):** Strategia in cui si processa la radice immediatamente all'atto della scoperta, per poi procedere iterativamente all'attraversamento del sottoalbero sinistro e successivamente del sottoalbero destro.

[5]**Visita in Ordine Simmetrico (In-Order):** Strategia (tipica degli alberi binari) che esplora totalmente il sottoalbero sinistro, elabora la radice come spartiacque, e infine esplora il sottoalbero destro.

[6]**Visita in Ordine Posticipato (Post-Order):** Strategia "bottom-up" in cui l'elaborazione del nodo padre è subordinata all'avvenuto esaurimento della visita di tutti i suoi sottoalberi (prima il sinistro, poi il destro).

[7]**Visita per Livelli:** Attraversamento topologico orizzontale, equivalente a una Visita in Ampiezza (BFS) governata da una struttura a Coda.

- *In-Order (Simmetrico):* Passate dal figlio sinistro, poi salutate il padre, poi andate al figlio destro. Se l'albero è un Albero Binario di Ricerca (BST), questa visita vi restituisce miracolosamente tutti i numeri in perfetto ordine crescente.

Analisi Passo-Passo: L'implementazione iterativa del Pre-Order con la Pila nasconde un trucco logico brillante spiegato a lezione:

1. Mettiamo la Radice nello Stack.
2. Togliamo la Radice e la stampiamo. Ora dobbiamo mettere i suoi due figli (Sinistro e Destro) nello Stack.
3. Poiché vogliamo che il Sinistro venga stampato subito alla prossima interazione, e lo Stack fa uscire per prima l'ultima cosa entrata (LIFO), dobbiamo fare `Push(Figlio DESTRO)` e poi `Push(Figlio SINISTRO)`. Invertire l'ordine di ingresso garantisce l'ordine corretto di uscita.

Il "Perché" è importante: Conoscere la differenza spaziale $O(n)$ ci allerta su un problema sistemico. Se l'albero è un "albero degenere" (tutto storto da un lato, simile a un fagiolo rampicante), la memoria si riempirà rapidamente fino a n. Se l'albero è bilanciato, lo stack occuperà solo uno spazio ridicolo, pari a $\approx \log_2(n)$.

Ripasso Visivo/Schematico:

Strategia	**Ordine di Stampa**	**Memoria Usata**	**Impiego Tipico**
Pre-Order	Padre $\rightarrow$ Sinistro $\rightarrow$ Destro	Stack (LIFO)	Duplicazione/Serializzazione alberi.
In-Order	Sinistro $\rightarrow$ Padre $\rightarrow$ Destro	Stack (LIFO)	Ordinamento di valori (su Alberi BST).
Post-Order	Sinistro $\rightarrow$ Destro $\rightarrow$ Padre	Stack (con Flag)	Calcolo delle dimensioni (Bottom-Up).
Level-Order	Scansione orizzontale (piani)	Coda (FIFO)	Ricerca del nodo più vicino.

Capitolo 16

Visita Iterativa di Alberi e Problema dell'Ordinamento

30/03/2026 Visite Iterative: Ordine Simmetrico e Posticipato

Output 1: Diario Generale di Ripasso (Formato Accademico)

- **Struttura di un problema:** Eliminazione della ricorsione nativa nell'attraversamento strutturale degli alberi binari, simulando esplicitamente la memoria di lavoro (Stack/Pila LIFO) per garantire le corrette priorità di ispezione dei sottoalberi (In-Order e Post-Order).
- **Partiamo da una definizione formale:**
  - **Visita in Ordine Simmetrico (In-Order)**[1]
  - **Visita in Ordine Posticipato (Post-Order)**[2]
  - **Flag di Stato (True/False)**[3]
- **La soluzione:**
  - **Algoritmo In-Order Iterativo:** L'esecuzione è governata da un ciclo `while`. Dato un nodo di partenza, lo si inserisce nello Stack e si scende progressivamente verso sinistra, allocando nello Stack tutti i nodi incontrati fino a raggiungere un riferimento nullo (`nil`). A questo punto si estrae l'elemento in cima alla Pila (che sarà il nodo più a sinistra privo di figli sinistri), lo si visita, e si tenta una transizione verso il suo figlio destro, ripetendo l'intero ciclo.
  - **Algoritmo Post-Order Iterativo:** La gestione della Pila richiede l'uso combinato di nodi e di un *Flag di Stato*.
    1. Si inizializza lo Stack inserendo la radice con etichetta `True`.

[1] **Visita in Ordine Simmetrico (In-Order):** Esplorazione iterativa che impone di esaurire completamente la scansione del ramo sinistro di un nodo prima di processare il nodo stesso, delegando la visita del ramo destro al passo logicamente successivo.

[2] **Visita in Ordine Posticipato (Post-Order):** Esplorazione iterativa in cui l'elaborazione della radice è strettamente subordinata alla preventiva e completa visita di entrambi i sottoalberi dipendenti (sinistro e poi destro).

[3] **Flag di Stato (True/False):** Etichetta booleana associata dinamicamente a ciascun nodo inserito nello Stack durante la visita Post-Order per distinguere una "prenotazione di visita dei figli" (*True*) dall'effettiva "autorizzazione alla stampa del nodo" (*False*).

2. Si estrae il nodo Top. Se la sua etichetta è `True`, l'esecutore lo reinserisce immediatamente nello Stack mutando il suo stato in `False` (prenotazione alla stampa differita). Successivamente, inserisce i figli nello Stack (prima il destro, poi il sinistro) associando loro l'etichetta `True`.
3. Se il nodo estratto possiede etichetta `False` (oppure è una foglia priva di figli), viene definitivamente visitato e rimosso dalla memoria.

Guida di Studio Approfondita e Semplificata (Formato Apprendimento)

Il Concetto Base: Trasformare una funzione ricorsiva in iterativa significa dover fare a mano ciò che il computer di solito fa in automatico dietro le quinte.

- Nell'**Ordine Simmetrico**, il nostro esploratore si tuffa a capofitto sempre a sinistra, riempiendo lo zaino (la Pila) con tutti i bivi che incontra. Quando sbatte contro un muro, tira fuori l'ultimo nodo dallo zaino, lo analizza, e poi fa un passo a destra per ricominciare a tuffarsi a sinistra.

- Nell'**Ordine Posticipato**, la faccenda è più complessa: il padre deve aspettare che i figli finiscano. Ma come fa la Pila a ricordarsi se il padre sta aspettando o se ha appena iniziato? Usiamo un "bigliettino" (il Flag). Se ha su scritto *True*, significa "Guarda prima i miei figli e rimettimi in fondo ad aspettare". Se c'è scritto *False*, significa "I miei figli hanno finito, finalmente tocca a me!".

Analisi Passo-Passo (Il trucco del Post-Order): L'inversione logica è geniale. Estraggo un nodo con *True*. Invece di stamparlo, lo rimetto dentro con *False*. Poi "schiaccio" sopra di lui il figlio destro (*True*) e infine il figlio sinistro (*True*). Dato che la Pila funziona al contrario (Last In, First Out), al prossimo giro uscirà per primo proprio il figlio sinistro, garantendo esattamente l'ordine di visita desiderato.

Il "Perché" è importante: Gli algoritmi iterativi, pur essendo più lunghi da scrivere e richiedendo una Pila esplicita, prevengono il temuto *Stack Overflow* di sistema tipico delle ricorsioni profonde. Questo è essenziale nella programmazione di sistemi critici o nell'analisi di alberi generati da enormi basi di dati strutturate.

Ripasso Visivo/Schematico:

Visita	**Meccanismo della Pila**	**Azione alla Pop (Estrazione)**
In-Order	Discesa iterativa a sinistra.	Visita del nodo, poi transizione immediata al figlio destro.
Post-Order	Sistema a Flag doppio stato (`True/False`).	Se `True`: reinserisce con `False` e impila i figli. Se `False`: visita.

30/03/2026 Introduzione al Problema dell'Ordinamento

Output 1: Diario Generale di Ripasso (Formato Accademico)

- **Struttura di un problema:** Riorganizzazione posizionale di una collezione di dati (astratta come sequenza) in ottemperanza a una specifica relazione d'ordine totale, distinguendo gli approcci operativi in funzione delle limitazioni hardware e della natura delle chiavi.

- **Partiamo da una definizione formale:**

- **Problema dell'Ordinamento**[4]
- **Ordinamento Interno vs Esterno**[5]
- **Algoritmi per Confronti e Scambi**[6]
- **Algoritmi Digitali**[7]

- **La soluzione:**
  - L'analisi del problema dell'ordinamento si astrae dall'implementazione hardware (vettore vs lista concatenata) definendo le operazioni su una "sequenza".
  - Ciononostante, il rendimento dell'algoritmo subirà in molti casi degradazioni asintotiche se mappato su una struttura ad accesso strettamente sequenziale (lista) piuttosto che ad accesso casuale (vettore), a causa del costo di individuazione posizionale.

Guida di Studio Approfondita e Semplificata (Formato Apprendimento)

Il Concetto Base: Ordinare dei dati sembra banale (mettere in fila i numeri dal più piccolo al più grande), ma per un computer è un'operazione estremamente dispendiosa. Ci sono due grandi famiglie di "filosofie" per ordinare:

1. *Per confronti:* È come mettere in ordine di altezza una classe di bambini confrontandoli spalla contro spalla due alla volta.

2. *Metodi digitali:* Invece di confrontare due bambini tra loro, chiedo a tutti di dirmi la loro età e li smisto in "scatoloni" separati (quelli di 6 anni, quelli di 7 anni...). Evito di confrontarli fisicamente tra loro!

Analisi Passo-Passo (Interno vs Esterno): Se dobbiamo ordinare una cartella con 100 foto, possiamo aprirle tutte sul desktop (Memoria RAM): questo è l'**Ordinamento Interno**. È iper-veloce. Se dobbiamo ordinare tutti i dati anagrafici dei cittadini europei (Terabyte di dati), il computer esploderebbe provando a caricarli tutti nella RAM limitata (es. 16 GB). È obbligato a caricare "fette" di dati dal disco fisso, ordinarle, salvarle, e caricarne di nuove. Questo è l'**Ordinamento Esterno** e richiede algoritmi specifici e molto cauti nell'uso dei salvataggi su disco.

Il "Perché" è importante: Il problema dell'ordinamento è uno dei pilastri centrali dell'informatica teorica e pratica. Disporre di dati ordinati abbatte drasticamente i tempi di ricerca successiva (permettendo la Ricerca Binaria logaritmica invece della lentissima Ricerca Sequenziale lineare).

Ripasso Visivo/Schematico:

[4]**Problema dell'Ordinamento:** Sia U un insieme di elementi e $\leq$ una relazione d'ordine totale su U (riflessiva, transitiva, antisimmetrica). Data un'istanza $X \in U^n$, generare una sequenza in uscita in cui $X[i] \leq X[i+1]$ per ogni $1 \leq i < n$.

[5]**Ordinamento Interno vs Esterno:** L'ordinamento è *Interno* qualora l'intera sequenza X sia dislocabile contestualmente nella memoria centrale (RAM). È *Esterno* quando l'istanza eccede i limiti fisici della RAM, imponendo il ricorso a memorie di massa (disco) e tecniche a blocchi.

[6]**Algoritmi per Confronti e Scambi:** Famiglia di algoritmi la cui logica si fonda unicamente sulla comparazione atomica tra due valori ($X[i] \leq X[j]$) e il conseguente scambio di posizione. Limite asintotico inferiore matematico $\Omega(n \log n)$.

[7]**Algoritmi Digitali:** Approcci che eludono il confronto diretto operando sulla morfologia binaria o strutturale della chiave (es. ispezione lessicografica, counting).

Categoria	Caratteristica Operativa	Dipendenza dal Dato
Confronti e Scambi	Compara coppie e scambia posizioni.	Universale, funziona su qualsiasi tipo di dato confrontabile.
Digitali	Ispeziona i singoli bit o caratteri.	Specifico, dipende dalla natura costruttiva della chiave.

30/03/2026 Proprietà degli Algoritmi: Stabilità e Adattività

Output 1: Diario Generale di Ripasso (Formato Accademico)

- **Struttura di un problema:** Categorizzare gli algoritmi di ordinamento non solo in virtù della loro complessità asintotica, ma in base alla loro sensibilità rispetto alla topologia preesistente dei dati di ingresso e al trattamento delle collisioni logiche (chiavi identiche).
- **Partiamo da una definizione formale:**
  - **Stabilità**[8]
  - **Adattività**[9]
- **La soluzione:**
  - **Gestione della Stabilità:** Cruciale nei processi di "ordinamento a più chiavi". Ordinando ad esempio un database prima per *Nome* e in successione per *Anno di Immatricolazione*, un algoritmo instabile distruggerebbe l'ordinamento alfabetico formatosi all'interno del sottogruppo di studenti iscritti nello stesso anno.
  - **Riconoscimento dell'Adattività:** Se l'algoritmo adotta costrutti di ispezione condizionale capaci di indurre interruzioni precoci dei cicli (*early exit*) qualora una porzione dell'array risulti localmente ordinata, il metodo è adattivo. In caso contrario (l'algoritmo compie passaggi forzati blindati su for-loop deterministici), il costo esecutivo collassa sul caso peggiore anche in presenza di array già ordinati.

Guida di Studio Approfondita e Semplificata (Formato Apprendimento)

Il Concetto Base: Immaginate di avere due carte da gioco con lo stesso valore: un 7 di Cuori e un 7 di Picche. Nel mazzo scombinato, il Cuori viene prima del Picche. Se l'algoritmo, alla fine del riordino, vi restituisce ancora il 7 di Cuori prima del 7 di Picche, è un algoritmo **Stabile**. Se li inverte casualmente, è **Instabile**. L'**Adattività** invece è l'intelligenza dell'algoritmo di accorgersi se "metà del lavoro è già fatto". Se gli date un mazzo di carte già quasi totalmente ordinato, un algoritmo adattivo finisce in una frazione di secondo. Uno non adattivo esegue meccanicamente tutti i controlli uno per uno impiegando lo stesso tempo di un mazzo caotico.

Analisi Passo-Passo: Perché la Stabilità è vitale nel mondo reale? I database moderni (come Excel o SQL) spesso fanno ordinamenti multipli. Volete ordinare i dipendenti per Età, ma a parità di Età volete che siano in ordine Alfabetico.

[8] **Stabilità:** Un algoritmo di ordinamento è detto *stabile* se e solo se, per ogni coppia di elementi x_i, x_j presenti in input con il medesimo valore di chiave ($Val(x_i) = Val(x_j)$) e dove x_i precede x_j spazialmente nell'input, x_i continuerà a precedere x_j nell'output ordinato.

[9] **Adattività:** Un algoritmo è detto *adattivo* qualora il numero totale di confronti eseguiti durante l'elaborazione non sia costante, bensì dipenda in modo funzionale dal grado di pre-ordinamento (o numero di inversioni) dell'istanza in input.

1. Si ordina tutto il database in ordine Alfabetico.
2. Si applica un secondo algoritmo *Stabile* per ordinare l'Età.

Il database è ora perfetto: tutti i 30enni sono in un blocco e, grazie alla stabilità dell'algoritmo, non avendo rimescolato i "doppioni di età", all'interno dei 30enni l'ordine alfabetico precedente è rimasto intatto!

Il "Perché" è importante: Saper riconoscere queste due etichette permette al progettista software di selezionare l'utensile giusto: algoritmi instabili e non adattivi (come alcune implementazioni basi del Selection Sort) sono pessimi per applicazioni di database sequenziali, ma magari sono scelti in contesti hardware in cui la limitazione della memoria e la semplicità dei cicli è la priorità assoluta.

Ripasso Visivo/Schematico:

Proprietà	**Azione**	**Vantaggio Pratico**
Stabile	Rispetta l'ordine di arrivo dei doppioni (chiavi uguali).	Essenziale per l'ordinamento concatenato su più campi (es. Cognome $\rightarrow$ Età).
Adattivo	Riduce dinamicamente i confronti se i dati sono parzialmente ordinati.	Massimizza l'efficienza temporale in scenari reali di aggiornamento dati pre-ordinati.

Lezioni da aggiungere:

- 31/03/2026
- 9/04/2026
- 13/04/2026
- 14/04/2026
- 16/04/2026
- 20/04/2026
- 21/04/2026
- 23/04/2026